

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики
Дисциплина: Компьютерные системы и сети

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе
на тему
**ВЕБ-ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ ИГРУ
«ТЕХАССКИЙ ХОЛДЕМ»**

БГУИР КП 6-05-0612-01 018 ПЗ

Студент

Я. В. Калач

Руководитель

С. В. Болтак

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Анализ предметной области	6
1.1 Сравнительный анализ существующих решений.....	6
1.2 Постановка задачи	8
2 Проектирование программы	10
2.1 Проектирование структуры программы	10
2.2 Проектирование системы сетевого взаимодействия.....	11
3 Разработка программного средства.....	15
3.1 Описание разработанных модулей.....	15
3.2 Описание сетевого модуля	17
4 Тестирование	20
5 Руководство пользователя.....	27
Заключение	35
Список используемых источников	36
Приложение А: Листинг кода с комментариями	37
Приложение Б: Схема алгоритма работы системы.....	65

ВВЕДЕНИЕ

Современные веб-технологии позволяют создавать интерактивные многопользовательские приложения, работающие непосредственно в браузере без установки дополнительного программного обеспечения. Одним из востребованных направлений являются сетевые игры, обеспечивающие взаимодействие пользователей в режиме реального времени через Интернет. Разработка подобных систем требует применения современных средств клиент-серверного взаимодействия, организации обмена данными и реализации игровой логики.

Веб-приложение, реализующее сетевую игру «Техасский холдем», представляет собой программный комплекс клиент-серверной архитектуры, предназначенный для организации игровых комнат, взаимодействия игроков и проведения полноценных покерных партий в режиме реального времени.

Целью проекта является разработка полнофункционального веб-приложения для организации многопользовательской сетевой игры в покер. Разрабатываемая система должна обеспечивать стабильное взаимодействие игроков в режиме реального времени, реализовывать игровую логику покера, а также предоставлять удобный и отзывчивый пользовательский интерфейс.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить предметную область и существующие аналоги;
- спроектировать архитектуру клиент-серверного приложения и определить механизм взаимодействия между клиентской и серверной частями;
- выбрать технологический стек для разработки веб-приложения;
- реализовать систему регистрации и аутентификации пользователей;
- разработать игровую логику покера [1], включающую управление игровыми комнатами, обработку действий игроков, раздачу карт, расчет банков и определение победителей;
- разработать клиентскую часть приложения на HTML, CSS и JavaScript с отображением игрового стола, карт, действий игроков и состояния партии;
- провести тестирование разработанного программного обеспечения.

Актуальность разработки обусловлена широким распространением браузерных многопользовательских игр без установки специализированного программного обеспечения. Использование технологий WebSocket и Socket.IO позволяет организовать двусторонний обмен данными между клиентом и сервером с минимальными задержками. Дополнительно применение современных средств веб-разработки, таких как Node.js, Express, PostgreSQL и JWT-аутентификация, обеспечивает масштабируемость, безопасность и удобство сопровождения разработанной системы.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Сравнительный анализ существующих решений

Для проведения сравнительного анализа были выбраны три популярных веб-ресурса, реализующих многопользовательскую игру в покер в браузере.

«PokerStars» является одной из крупнейших платформ для онлайн-покера [2]. Сервис предоставляет широкий выбор игровых режимов, систему турниров, рейтингов и статистики игроков. Реализована поддержка многопользовательских столов, внутриигрового чата и системы авторизации пользователей. Клиентская часть выполнена в современном стиле и поддерживает работу как на персональных компьютерах, так и на мобильных устройствах. Вместе с тем платформа ориентирована преимущественно на коммерческое использование и требует обязательной регистрации пользователя, а часть функционала доступна только через отдельное клиентское приложение. На рисунке 1.1 представлен интерфейс платформы PokerStars.



Рисунок 1.1 – Интерфейс платформы PokerStars

«247freepoker.com» представляет собой браузерную реализацию покера [3], разработанную с использованием HTML5. Платформа позволяет играть без установки дополнительного программного обеспечения и отличается простым пользовательским интерфейсом. Реализована игра против

компьютерных соперников, однако многопользовательский режим отсутствует. Несмотря на удобство и адаптивность интерфейса, отсутствие сетевого взаимодействия значительно ограничивает функциональные возможности ресурса. На рисунке 1.2 представлен интерфейс платформы 247freepoker.com.



Рисунок 1.2 – Интерфейс платформы 247freepoker.com

«Replay Poker» является веб-платформой для многопользовательской игры в покер [4]. Сервис поддерживает игровые столы для нескольких участников, систему виртуальных фишек, турниры и статистику игроков. Для взаимодействия пользователей используется технология WebSocket, обеспечивающая обновление игрового состояния в режиме реального времени. Интерфейс платформы выполнен в современном стиле и корректно работает в браузере без необходимости установки клиента. Однако значительная часть функционала ориентирована на крупные игровые сообщества и содержит избыточные элементы, не являющиеся обязательными для базовой реализации сетевой карточной игры. На рисунке 1.3 представлен интерфейс платформы Replay Poker.



Рисунок 1.3 – Интерфейс платформы Replay Poker

Таким образом, проведённый сравнительный анализ позволяет определить основные требования к разрабатываемому программному средству. PokerStars предоставляет широкий функционал, однако ориентирован на коммерческое использование и требует сложной инфраструктуры. 247freepoker.com не поддерживает многопользовательский режим. Replay Poker обладает современным интерфейсом и сетевым взаимодействием, но содержит избыточный функционал. Разрабатываемое приложение занимает нишу лёгкого веб-приложения для многопользовательской игры в покер с использованием технологий WebSocket и Socket.IO, обеспечивающих взаимодействие игроков в режиме реального времени.

1.2 Постановка задачи

Данная курсовая работа посвящена разработке веб-приложения, реализующего сетевую карточную игру «Техасский холдем». Основная цель проекта заключается в создании программного средства, обеспечивающего взаимодействие нескольких игроков в режиме реального времени через

браузер без необходимости установки дополнительного программного обеспечения.

Разрабатываемое приложение построено на клиент-серверной архитектуре. Взаимодействие между клиентской и серверной частями осуществляется по протоколу WebSocket с использованием библиотеки Socket.IO [5], что позволяет оперативно синхронизировать состояние игры между всеми участниками партии.

Программа должна обеспечивать следующие функциональные возможности:

- регистрацию и авторизацию пользователей;
- создание игровых столов с заданными параметрами;
- подключение игроков к существующим игровым комнатам;
- отображение списка доступных игровых столов;
- проведение полного игрового процесса согласно правилам покера «Техасский холдем»;
- автоматическую раздачу карт игрокам и общих карт на стол;
- выполнение игровых действий (Fold, Check, Call, Raise, All-in);
- проведение раундов торговли и переход между стадиями игры (префлоп, флоп, терн, ривер);
- вычисление покерных комбинаций и определение победителя раздачи;
- подсчёт и распределение банка между победителями;
- ведение статистики игроков и хранение информации о балансе фишек;
- обработку отключений пользователей и возможность восстановления игровой сессии после переподключения.

К нефункциональным требованиям системы относятся:

- обеспечение обмена игровыми событиями в режиме реального времени с минимальными задержками;
- корректная работа приложения в современных веб-браузерах;
- валидация всех игровых действий на стороне сервера;
- сохранение данных пользователей и игровых сессий в базе данных PostgreSQL [6];
- обеспечение безопасности хранения учётных данных пользователей.

Таким образом, разрабатываемое программное средство должно обеспечивать полноценную многопользовательскую игру в покер «Техасский холдем» через веб-интерфейс, предоставляя пользователям удобные средства взаимодействия и надёжную синхронизацию игрового процесса в режиме реального времени.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММЫ

2.1 Проектирование структуры программы

При проектировании веб-приложения была выбрана двухуровневая клиент-серверная архитектура. Система состоит из клиентской и серверной частей, взаимодействующих между собой через сеть Интернет. Такое разделение позволяет разграничить ответственность между компонентами приложения: клиент отвечает за отображение интерфейса и взаимодействие с пользователем, а сервер обеспечивает обработку игровой логики, хранение данных и контроль корректности игровых действий.

Выбранная архитектура обладает рядом преимуществ. Во-первых, все критически важные вычисления выполняются на сервере, что исключает возможность изменения игрового состояния со стороны клиента. Во-вторых, разделение приложения на независимые компоненты упрощает сопровождение и дальнейшее развитие проекта. В-третьих, такая структура позволяет одновременно обслуживать несколько игровых столов и большое количество пользователей.

Структура серверной части включает несколько функциональных модулей:

- модуль управления пользователями обеспечивает регистрацию и авторизацию игроков, проверку учётных данных, генерацию и проверку JWT-токенов, а также получение информации о профиле пользователя;
- модуль управления игровыми комнатами отвечает за создание, хранение и удаление игровых столов, подключение и отключение игроков, а также контроль состояния игровых сессий;
- модуль игровой логики является центральным элементом системы. Он реализует правила игры «Техасский холдем», управляет этапами раздачи, обработкой действий игроков, определением очередности ходов, вычислением победителей и распределением банка между участниками;
- модуль работы с картами реализует создание и перемешивание колоды, раздачу карт игрокам и формирование общих карт стола;
- модуль оценки комбинаций предназначен для анализа карт игроков и определения силы покерных комбинаций согласно правилам игры. Результаты его работы используются при определении победителя раздачи;
- модуль управления банком обеспечивает учёт ставок игроков, формирование основного и дополнительных банков, а также распределение выигрыша между победителями;
- модуль хранения состояния игры отвечает за сохранение текущего состояния игровых столов и восстановление данных при необходимости переподключения пользователей.

Для хранения информации о пользователях, статистике игр и балансе фишек используется система управления базами данных PostgreSQL.

Клиентская часть представляет собой веб-приложение, работающее в браузере пользователя. Она включает несколько основных модулей:

- модуль аутентификации обеспечивает регистрацию, вход в систему и управление пользовательской сессией;
- модуль игрового интерфейса отвечает за отображение покерного стола, карт игроков, общих карт, банка и элементов управления игровым процессом;
- модуль управления действиями игрока предоставляет возможность выполнения игровых действий, таких как Fold, Check, Call, Raise и All-in, а также контролирует доступность этих действий в зависимости от состояния партии;
- модуль отображения состояния игры обеспечивает визуализацию информации об игроках, размере их стеков, текущем ходе, последних действиях участников и результатах раздачи;
- модуль управления комнатами реализует создание игровых столов, отображение списка доступных комнат и подключение пользователей к выбранной игровой сессии.

Таким образом, структура программы построена по модульному принципу, что обеспечивает разделение функциональности между компонентами системы, упрощает сопровождение программного обеспечения и позволяет расширять функциональные возможности приложения без существенного изменения существующей архитектуры.

2.2 Проектирование системы сетевого взаимодействия

Для обеспечения взаимодействия между участниками игрового процесса в разрабатываемом приложении была выбрана модель клиент-серверного взаимодействия. Все игровые события обрабатываются сервером, который выступает в роли централизованного узла управления игровыми сессиями и поддерживает единое актуальное состояние игры для всех подключённых пользователей.

Система сетевого взаимодействия основана на использовании двух протоколов: HTTP и WebSocket. Протокол HTTP применяется для выполнения вспомогательных операций, не требующих постоянного соединения с сервером. К таким операциям относятся регистрация и авторизация пользователей, получение информации о профиле игрока, получение списка игровых столов и создание новых игровых комнат.

Для обновления информации в лобби приложения используется технология Server-Sent Events (SSE) [7]. Данная технология обеспечивает одностороннюю передачу данных от сервера к клиенту по постоянному HTTP-соединению и позволяет получать изменения состояния игровых столов без выполнения периодических запросов.

После открытия страницы лобби клиент устанавливает SSE-соединение с сервером и подписывается на получение событий обновления списка игровых комнат. При создании нового стола, подключении игрока или удалении комнаты сервер автоматически отправляет обновлённые данные всем подключённым клиентам. Благодаря этому пользователи всегда получают актуальную информацию о доступных игровых столах в режиме реального времени.

Для организации данного механизма реализован эндпоинт:

GET /api/tables/events – подписка на обновления списка игровых столов посредством технологии SSE.

Использование SSE позволяет снизить нагрузку на сервер по сравнению с периодическим опросом REST API и избежать избыточного применения WebSocket-соединений для задач [8], не требующих двустороннего обмена данными.

После подключения пользователя к игровой комнате дальнейшее взаимодействие осуществляется посредством протокола WebSocket, реализованного с использованием библиотеки Socket.IO. Данная технология обеспечивает постоянное двустороннее соединение между клиентом и сервером, позволяя передавать игровые события практически без задержек. Использование WebSocket исключает необходимость периодического опроса сервера и снижает сетевую нагрузку по сравнению с традиционными HTTP-запросами.

При подключении к игровому столу клиент устанавливает WebSocket-соединение с сервером и передаёт идентификатор игровой комнаты. Сервер регистрирует подключение пользователя и добавляет его в соответствующую игровую сессию. После подключения всех участников сервер инициирует начало игровой партии и рассылает клиентам начальное состояние игры.

В процессе игры через WebSocket передаются следующие категории сообщений:

- подключение и отключение игроков;
- начало новой раздачи;
- раздача карт игрокам;
- обновление состояния игрового стола;
- выполнение игровых действий (Fold, Check, Call, Raise, All-in);
- переход между этапами игры (префлоп, флоп, терн, ривер);
- завершение раздачи и определение победителя;
- обновление информации о банке и количестве фишек игроков;
- уведомления о переподключении пользователя и восстановлении игровой сессии.

Для обмена данными используется формат JSON. Каждое сообщение содержит тип события и набор параметров, необходимых для его обработки. Применение JSON обеспечивает простоту сериализации данных и независимость от используемых языков программирования на стороне клиента и сервера. Основные типы сообщений представлены в таблице 2.1.

Таблица 2.1 – Типы сообщений Socket.IO

Тип сообщения	Направление	Описание
create-room	Клиент → Сервер	Создание игрового стола
join-room	Клиент → Сервер	Подключение к игровому столу
leave-room	Клиент → Сервер	Выход из игрового стола
player-action	Клиент → Сервер	Выполнение игрового действия (Fold, Check, Call, Raise, All-in)
get-game-state	Клиент → Сервер	Получение актуального состояния игры
ready	Клиент → Сервер	Подтверждение готовности к началу новой раздачи
game-state	Сервер → Клиент	Передача состояния игрового стола
hole-cards	Сервер → Клиент	Передача карманных карт игроку
hand-started	Сервер → Клиент	Начало новой раздачи
blinds-posted	Сервер → Клиент	Информация о блайндах
street-changed	Сервер → Клиент	Переход между стадиями торговли
player-turn	Сервер → Клиент	Передача права хода активному игроку
account-chips	Сервер → Клиент	Обновление баланса фишек
session-kicked	Сервер → Клиент	Принудительное завершение пользовательской сессии

Для обеспечения корректности игрового процесса все действия игроков проходят обязательную серверную проверку. Клиентское приложение выполняет только отображение интерфейса и отправку запросов на выполнение действий. Проверка очередности хода, достаточности количества фишек, допустимости ставок и корректности переходов между этапами игры осуществляется исключительно на стороне сервера. Такой подход

предотвращает возможность нарушения игровых правил и обеспечивает единое состояние игры для всех участников.

С целью повышения надёжности работы системы предусмотрена обработка временного разрыва соединения. При потере связи клиент автоматически выполняет повторное подключение к серверу. После успешного восстановления соединения сервер отправляет актуальное состояние игрового стола, что позволяет игроку продолжить участие в партии без потери данных.

Спроектированная система сетевого взаимодействия обеспечивает передачу игровых событий в режиме реального времени, поддерживает синхронизацию состояния игры между всеми участниками и создаёт необходимые условия для стабильной работы многопользовательского веб-приложения.

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Описание разработанных модулей

В ходе разработки программного средства были реализованы модули клиентской и серверной частей, обеспечивающие функционирование многопользовательской игры в покер «Техасский холдем». Каждый модуль отвечает за выполнение определённого набора задач и взаимодействует с другими компонентами системы через определённые интерфейсы.

Модуль управления пользователями и сессиями реализует регистрацию, аутентификацию игроков и непрерывное сопровождение пользовательских сессий. Функциональность модуля распределена между клиентскими скриптами и серверными контроллерами.

На стороне сервера (компонент `authRoutes.js`) модуль обрабатывает HTTP-запросы регистрации и входа. При регистрации выполняется проверка уникальности имени пользователя в базе данных PostgreSQL, а пароли хэшируются с помощью библиотеки `bcrypt` перед сохранением [9]. При успешной авторизации сервер генерирует сессионный JWT-токен [10], содержащий идентификатор, имя и баланс фишек игрока.

На стороне клиента взаимодействие обеспечивают три ключевых компонента:

- `forms.js` – перехватывает данные из экранных форм, осуществляет их валидацию (логин от 3 до 20 символов, пароль от 6 символов), отправляет запросы на сервер через `fetch API` и динамически обновляет элементы навигации UI;

- `session.js` – инкапсулирует логику сохранения, чтения и удаления JWT-токена и профиля пользователя в локальном хранилище браузера (`localStorage`), а также синхронизирует баланс фишек через эндпоинт `/api/profile/me`;

- `sessionMonitor.js` – непрерывно отслеживает валидность токена и предотвращает одновременный вход с нескольких устройств. В случае фиксации повторной авторизации или истечения сессии компонент очищает `localStorage` и выполняет принудительное перенаправление на страницу входа.

Модуль доступа к данным обеспечивает взаимодействие серверной части с базой данных PostgreSQL. Через данный модуль выполняются операции сохранения и получения информации о пользователях, балансе игровых фишек, статистике партий и истории игровых действий. Использование отдельного слоя доступа к данным позволило изолировать бизнес-логику от особенностей работы с системой управления базами данных.

Модуль управления игровыми комнатами координирует распределение участников по игровым столам и обеспечивает сетевое взаимодействие. Функциональность модуля распределена между клиентскими интерфейсами и серверным компонентом `roomManager.js`.

На стороне сервера (`roomManager.js`) – отвечает за создание, хранение и удаление игровых столов, генерируя для каждой комнаты уникальный

идентификатор. Компонент контролирует количество участников за столом (лимиты мест), обрабатывает их подключение и отключение, а также управляет жизненным циклом комнат и синхронизирует метаданные активных столов без задержек ввода-вывода.

Сетевое взаимодействие (server.js) – на базе библиотеки Socket.IO мгновенно транслирует события подключения/отключения игроков и координирует работу лобби, запрашивая актуальные списки доступных столов.

Модуль игровой логики является центральным элементом системы и инкапсулирует алгоритмы, реализующие правила игры «Техасский холдем». Модуль полностью абстрагирован от сетевых интерфейсов и функционирует как конечный автомат.

Программный движок – последовательно переключает игровые стадии (префлоп, флоп, терн, ривер, шоудаун), генерирует и перемешивает колоду, а также строго контролирует очередность ходов.

Валидация и учет состояний (gameState.js) – осуществляет обязательную серверную проверку корректности действий игроков (Fold, Check, Call, Raise, All-in), исключая возможность модификации игрового состояния с клиента. Данный компонент также отвечает за сериализацию текущего состояния столов в JSON для восстановления данных при переподключении пользователей.

Модуль работы с картами обеспечивает формирование игровой колоды, её перемешивание и раздачу карт участникам. Для случайного распределения карт используется алгоритм перемешивания, обеспечивающий равномерное распределение вероятностей появления каждой карты в колоде. Модуль также отвечает за открытие общих карт на соответствующих этапах игры.

Для определения победителя раздачи реализован модуль оценки комбинаций. Его задача заключается в анализе карманных и общих карт, формировании возможных комбинаций и выборе наиболее сильной согласно правилам покера. Результаты вычислений используются при сравнении игроков и распределении банка.

Модуль управления банком выполняет учёт ставок игроков на протяжении всей раздачи. В его обязанности входит формирование основного и побочных банков, расчёт размеров ставок, контроль достаточности количества фишек у игроков и распределение выигрыша после завершения партии.

Сетевое взаимодействие реализовано отдельным модулем обмена сообщениями на основе технологии Socket.IO. Модуль обеспечивает передачу игровых событий между клиентами и сервером в режиме реального времени. Через него выполняются подключение игроков к игровым комнатам, отправка игровых действий, получение обновлённого состояния игрового стола и синхронизация игровых данных между всеми участниками партии.

Для обновления информации в лобби реализован модуль событийного оповещения на основе технологии Server-Sent Events (SSE). Данный

компонент обеспечивает автоматическую передачу клиентам информации о создании, изменении и удалении игровых столов без необходимости периодического опроса сервера.

Клиентская часть приложения разработана с использованием javascript [11], CSS и HTML [12], [13]. Основу пользовательского интерфейса составляет модуль игрового стола, отвечающий за отображение карт игроков, общих карт, размера банка и доступных игровых действий. Интерфейс динамически обновляется при получении новых данных от сервера.

Модуль управления действиями игрока обеспечивает обработку пользовательского ввода и передачу выбранных действий на сервер. В зависимости от текущего состояния игры пользователю отображаются доступные варианты действий: Fold, Check, Call, Raise или All-in.

Модуль отображения состояния игры предназначен для визуализации информации о ходе партии. Он отображает текущую стадию раздачи, размер банка, количество фишек игроков, активного участника и результаты завершённых раздач.

Таким образом, разработанное программное средство состоит из набора взаимосвязанных модулей, каждый из которых отвечает за отдельный аспект функционирования системы. Модульная организация приложения обеспечивает удобство сопровождения, возможность дальнейшего расширения функциональности и высокую надёжность работы многопользовательской игровой системы.

3.2 Описание сетевого модуля

Сетевое взаимодействие в разработанном приложении реализовано на платформе Node.js с использованием библиотеки Socket.IO [14]. Данный модуль обеспечивает обмен данными между клиентами и сервером в режиме реального времени, а также отвечает за управление игровыми комнатами, синхронизацию состояния игры и передачу игровых событий между участниками партии.

Точкой входа серверной части является файл server.js, в котором выполняется создание HTTP-сервера, настройка Express-приложения [15], подключение Socket.IO и регистрация маршрутов REST API. После запуска приложения сервер начинает прослушивание входящих HTTP- и WebSocket-подключений.

Центральным компонентом сетевого взаимодействия является модуль управления комнатами roomManager.js. Данный модуль регистрирует обработчики Socket.IO и определяет логику взаимодействия клиентов с игровыми столами. После установления WebSocket-соединения сервер начинает отслеживать события, поступающие от клиента.

Для обработки запросов пользователей реализованы следующие обработчики событий:

– join-room – подключение пользователя к существующему игровому столу по идентификатору комнаты. После успешного подключения игрок получает текущее состояние игрового процесса;

– leave-room – выход пользователя из игровой комнаты. Сервер удаляет игрока из списка участников и обновляет состояние игрового стола;

– player-action – выполнение игрового действия. В зависимости от переданных параметров сервер обрабатывает действия Fold, Check, Call, Raise или All-in и передаёт результат в игровой движок;

– get-game-state – получение актуального состояния игры. Используется при подключении пользователя к существующей игровой сессии и при восстановлении соединения;

– ready – подтверждение готовности игрока к началу новой раздачи после завершения предыдущей.

После обработки входящих запросов сервер формирует события для передачи клиентам. Основным сообщением является game-state, содержащее полное описание текущего состояния игрового стола: информацию об игроках, размере банка, текущей стадии игры, количестве фишек и истории последних действий.

Для передачи специализированных игровых событий используются следующие сообщения:

– hole-cards – отправка игроку его карманных карт;

– hand-started – уведомление о начале новой раздачи;

– blinds-posted – информация о размещении малого и большого блайндов;

– street-changed – уведомление о переходе между игровыми стадиями (префлоп, флоп, терн, ривер);

– player-turn – уведомление о переходе права хода к очередному участнику;

– account-chips – обновление количества фишек пользователя;

– session-kicked – уведомление о завершении пользовательской сессии при входе в аккаунт с другого устройства.

Управление активными игровыми столами осуществляется посредством специального менеджера комнат. Для каждого стола хранится информация о подключённых игроках, текущем состоянии партии, размере банка, очередности ходов и стадии игрового процесса. При получении нового события соответствующее состояние изменяется и автоматически синхронизируется между всеми участниками через Socket.IO.

Для обновления информации в игровом лобби реализован механизм Server-Sent Events (SSE). В отличие от WebSocket, данная технология обеспечивает одностороннюю передачу данных от сервера к клиенту и используется для оповещения пользователей об изменениях списка доступных игровых столов.

Подписка на события осуществляется посредством эндпоинта GET /api/session/stream. После установления соединения сервер сохраняет объект

подключения и может передавать клиенту уведомления о состоянии пользовательской сессии и изменениях, связанных с авторизацией.

REST API используется для выполнения операций, не требующих постоянного соединения. Через HTTP-запросы реализованы регистрация пользователей, авторизация, получение информации о профиле, управление балансом фишек и другие вспомогательные функции системы.

Для обеспечения безопасности сетевого взаимодействия все защищённые HTTP-запросы проходят проверку JWT-токена. После успешной аутентификации пользователь получает токен доступа, который передаётся серверу при выполнении последующих операций. Сервер проверяет корректность подписи токена и извлекает идентификатор пользователя, необходимый для выполнения запроса.

Таким образом, разработанный сетевой модуль обеспечивает взаимодействие пользователей в режиме реального времени, поддерживает синхронизацию состояния игровых столов, контроль пользовательских сессий и безопасный обмен данными между клиентской и серверной частями приложения.

4 ТЕСТИРОВАНИЕ

Тестирование является обязательным этапом разработки программного обеспечения, позволяющим проверить корректность реализации функциональных требований и выявить возможные ошибки до ввода системы в эксплуатацию. В рамках данного раздела представлены методика и результаты тестирования разработанного веб-приложения для многопользовательской игры в покер «Техасский холдем».

Целями тестирования являются:

- проверка корректности работы системы регистрации и авторизации пользователей;
- проверка создания и подключения к игровым столам;
- проверка корректности реализации игровых правил покера «Техасский холдем»;
- проверка надёжности сетевого взаимодействия через Socket.IO;
- проверка корректности работы механизма обновления лобби посредством Server-Sent Events;
- проверка обработки нестандартных ситуаций, включая отключение игрока и восстановление соединения.

Тестирование проводилось методом ручного функционального тестирования в условиях, приближённых к реальной эксплуатации. Испытания выполнялись в браузерах Google Chrome и Microsoft Edge под управлением операционной системы Windows 11.

Методика тестирования каждого сценария включала подготовку начального состояния системы, выполнение тестовых действий, сравнение фактического результата с ожидаемым и фиксацию результата проверки.

Результаты функционального тестирования представлены в таблице 4.1.

Таблица 4.1 – Результаты функционального тестирования

№	Тестируемая функция	Ожидаемый результат	Фактический результат	Статус
1	Регистрация пользователя	Создаётся новая учётная запись	Соответствует	Пройден
2	Авторизация пользователя	Пользователь успешно входит в систему	Соответствует	Пройден
3	Создание игрового стола	Создаётся новая игровая комната	Соответствует	Пройден
4	Подключение к игровому столу	Игрок успешно подключается к комнате	Соответствует	Пройден
5	Обновление списка столов через SSE	Изменения отображаются без	Соответствует	Пройден

№	Тестируемая функция	Ожидаемый результат	Фактический результат	Статус
		перезагрузки страницы		
6	Начало новой раздачи	Игрокам выдаются карты, назначаются блайнды	Соответствует	Пройден
7	Выполнение действия Fold	Игрок выходит из текущей раздачи	Соответствует	Пройден
8	Выполнение действия Check	Ход передаётся следующему игроку без изменения банка	Соответствует	Пройден
9	Выполнение действия Call	Размер ставки игрока выравнивается	Соответствует	Пройден
10	Выполнение действия Raise	Размер текущей ставки увеличивается	Соответствует	Пройден
11	Выполнение действия All-in	Все доступные фишки помещаются в банк	Соответствует	Пройден
12	Переход между стадиями игры	Корректный переход между префлопом, флопом, терном и ривером	Соответствует	Пройден
13	Раздача общих карт	Общие карты отображаются в соответствии с этапом игры	Соответствует	Пройден
14	Определение победителя	Победитель определяется по старшей комбинации	Соответствует	Пройден
15	Начисление выигрыша	Банк корректно распределяется между победителями	Соответствует	Пройден
16	Обновление баланса игрока	Количество фишек изменяется корректно	Соответствует	Пройден
17	Отключение игрока	Второй участник получает уведомление об отключении	Соответствует	Пройден
18	Повторное подключение	Игрок получает актуальное состояние игрового стола	Соответствует	Пройден
19	Защита от множественных сессий	Предыдущая сессия завершается при входе с другого устройства	Соответствует	Пройден

№	Тестируемая функция	Ожидаемый результат	Фактический результат	Статус
20	Проверка JWT-аутентификации	Запрос без токена отклоняется сервером	Соответствует	Пройден

По результатам тестирования все функциональные сценарии были успешно пройдены. В ходе разработки были выявлены и устранены следующие дефекты:

- некорректное обновление состояния игрового стола после выполнения некоторых игровых действий; проблема была устранена путём дополнительной синхронизации состояния между игровым движком и модулем Socket.IO;

- ошибки отображения баланса фишек после завершения раздачи; исправлены путём корректировки порядка обновления данных пользователя и отправки события account-chips.

Таким образом, результаты тестирования подтверждают корректность работы разработанного программного средства и соответствие реализованного функционала предъявляемым требованиям.

Также было проведено отслеживание пакетов в Wireshark.

Результат запроса основной страницы представлен на рисунке 4.2.

Аналогично работают запросы на другие страницы.

```

39 11.707270400  ::1          ::1          HTTP      900 GET / HTTP/1.1
41 11.724293400  ::1          ::1          HTTP      361 HTTP/1.1 304 Not Modified

```

Рисунок 4.2 – Запрос на получение основной страницы

Результат запроса на регистрацию /api/auth/register представлен на рисунке 4.3.

```

277 281.961172400  ::1          ::1          HTTP/J... 819 POST /api/auth/register HTTP/1.1 , JSON (application/json)
302 282.196048400  ::1          ::1          HTTP/J... 387 HTTP/1.1 201 Created , JSON (application/json)

```

Рисунок 4.3 – Запрос на регистрацию

Результат запроса с уже существующими данными /api/auth/register представлен на рисунке 4.4.

```

2465 1075.3846654... ::1          ::1          HTTP/J... 819 POST /api/auth/register HTTP/1.1 , JSON (application/json)
2486 1075.4342456... ::1          ::1          HTTP/J... 401 HTTP/1.1 409 Conflict , JSON (application/json)

```

Рисунок 4.4 – Ошибка регистрации

Результат запроса на логин /api/auth/login представлен на рисунке 4.5.

```

2576 1141.3619036... ::1 ::1 HTTP/J... 809 POST /api/auth/login HTTP/1.1 , JSON (application/json)
2601 1141.5458180... ::1 ::1 HTTP/J... 1061 HTTP/1.1 200 OK , JSON (application/json)

```

Рисунок 4.5 – Запрос на логин

Запрос на создание комнаты /api/rooms/create /api/rooms/create представлен на рисунке 4.6.

```

198 12.600120400 ::1 ::1 HTTP/J... 1180 POST /api/rooms/create HTTP/1.1 , JSON (application/json)
204 12.602967100 ::1 ::1 HTTP/J... 510 HTTP/1.1 201 Created , JSON (application/json)

```

Рисунок 4.6 – Запрос на создание комнаты

После успешного получения страницы комнаты устанавливается WebSocket соединение с применением socket.io (рисунок 4.7).

```

323 13.239386700 ::1 ::1 WebSoc... 71 WebSocket Text [FIN] [MASKED]

```

Рисунок 4.7 – WebSocket сообщение

При подключении к комнате после инициализации Вебсокета отправляется событие join-room (рисунок 4.8).

```

420["join-room",{"roomId":"room_1780507909966_65wmo","buyIn":100,"password":null}]

```

Рисунок 4.8 – Событие join-room

Событие подтверждения ready для начала игры (рисунок 4.9).

```

42["ready",{"roomId":"room_1780509341440_cnz65"}]

```

Рисунок 4.9 – Событие ready

Событие отправки игроку его карманных карт hole-cards (рисунок 4.10).

```

~ ~ ~ 42["hole-cards",{"holeCards":
:[{"rank":"K","suit":"s","value":13,"display":"K"},
{"rank":"6","suit":"h","value":6,"display":"6"}]]

```

Рисунок 4.10 – Событие hole-cards

Событие о размещении блайндов blind-posted (рисунок 4.11)

```
P42["blind-posted",{"seatId":8,"playerId":6,"smallBlind":5,"bigBlind":10}]]
```

Рисунок 4.11 – Событие blind-posted

Событие player-action для отправки своего действия (рисунок 4.12).

```
422["player-action",{"roomId":"room_1780509341440_cnz65","action":"call","amount":0}]]
```

Рисунок 4.12 – Событие player-action

Событие со сменой хода player-turn (рисунок 4.13)

```
~42["player-turn",{"playerId":6,"username":"Yarik2","toCall":0,"canCheck":true,"canCall":false,"canRaise":true,"minRaise":20,"maxRaise":100,"pot":20,"phase":"preflop","players":[{"seatId":0,"id":8,"username":"Yarik3","chips":90,"status":"active","bet":10,"totalContributed":10},{"seatId":1,"id":6,"username":"Yarik2","chips":90,"status":"active","bet":10,"totalContributed":10},null,null,null,null,null,null,null}]]
```

Рисунок 4.13 – Событие player-turn

Событие с обновлением стрита street-changed (рисунок 4.14).

```
~ 42[" street-c  
hanged", {"phase"  
:"flop", "communi  
tyCards": [{"rank"  
:"9", "suit": "h"  
,"value": 9, "disp  
lay": "9"}, {"r  
ank": "3", "suit":  
"d", "value": 3, "d  
isplay": "3"},  
{"rank": "9", "sui  
t": "d", "value": 9  
,"display": "9"}  
}]]]
```

Рисунок 4.14 – Событие street-changed

Событие раскрытие карт и определение победителя showdown (рисунок 4.15).

```
~ 42[" showdown  
", {"communityCar  
ds": [{"rank": "9"  
,"suit": "h", "val  
ue": 9, "display":  
"9"}, {"rank":  
"3", "suit": "d", "  
value": 3, "displa  
y": "3"}, {"ran  
k": "9", "suit": "d  
", "value": 9, "dis  
play": "9"}, {"  
rank": "5", "suit"  
:"h", "value": 5, "  
display": "5"},  
{"rank": "K", "su  
it": "h", "value":  
13, "display": "K  
"}, {"evaluation  
s": [{"playerId"  
: 8, "user name": "Y  
arik3", "holeCard  
s": [{"rank": "T",  
"suit": "h", "valu  
e": 10, "display":  
"10"}, {"rank":  
:"7", "suit": "h",  
"value": 7, "displ
```

Рисунок 4.15 – Событие showdown

Событие обновления фишек при выходе account-chips (рисунок 4.16).

```
42["account-chips",{"chips":1910}]
```

Рисунок 4.16 – Событие account-chips

Событие выхода из комнаты leave-room (рисунок 4.17).

```
426["leave-room",{"roomId":"room_1780509341440_cnz65"}]
```

Рисунок 4.17 – Событие leave-room

Событие принудительного завершения пользовательской сессии session-kicked (рисунок 4.18)

```
42["player-temporarily-disconnected",{"playerId":6,"username":"Yarik2","reconnectMs":15000}]
```

Рисунок 4.18 – Событие player-temporarily-disconnected

Событие определения победителя без определения победителя hand-ended-no-showdown (рисунок 4.19).

```
~ G42["hand-ended-no-showdown",  
{"winnerId":8,"winnerUsername":"Yarik3",  
"amount":15,"players":[{"seatIdx":0,"id":8,"username":"Yarik3",  
"chips":125,"status":"active",  
"bet":0,"totalContributed":0},  
{"seatIdx":1,"id":6,"username":"Yarik2",  
"chips":90,"status":"folded",  
"bet":0,"totalContributed":0}],  
null,null,null,null,null}]
```

Рисунок 4.19 – Событие hand-ended-no-showdown

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Данный раздел содержит описание порядка работы с веб-приложением «Техасский холдем» и предназначен для конечных пользователей системы.

Для начала работы с приложением необходимо открыть браузер и перейти по адресу развёрнутого веб-приложения. После загрузки страницы отображается меню (рисунок 5.1).

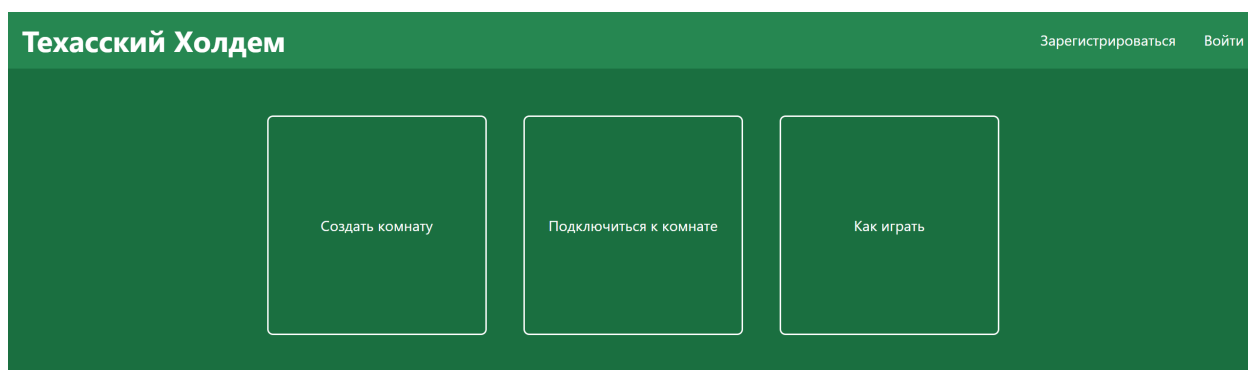


Рисунок 5.1 – Меню

Перед началом игры необходимо войти в аккаунт или зарегистрироваться. Вход/регистрация является обязательным пунктом, так как чтобы просмотреть профиль, создать комнату, подключиться к комнате без входа/регистрации нельзя. Чтобы зарегистрироваться нужно нажать на кнопку «Зарегистрироваться», далее ввести уникальный никнейм, пароль (от 6 символов включительно) и повторить пароль (рисунок 5.2).

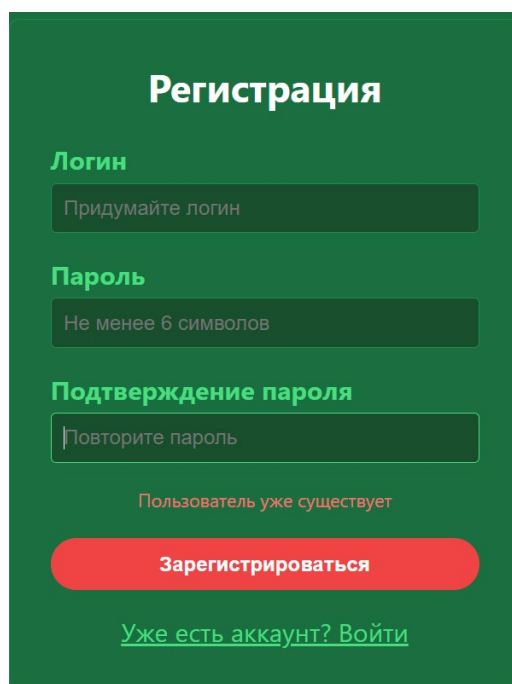
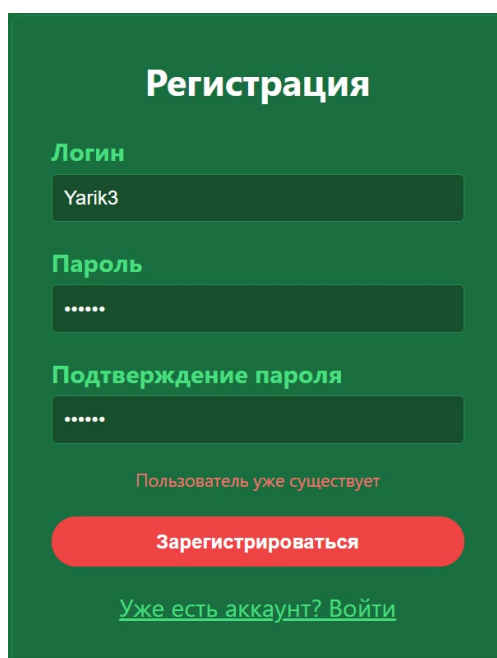


Рисунок 5.2 – Регистрация

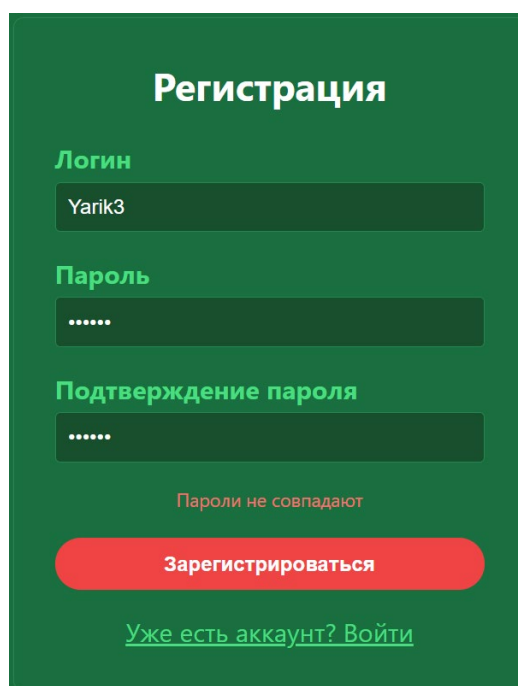
Если аккаунт с таким никнеймом существует, то высветится ошибка «Пользователь уже существует» (рисунок 5.3).



The screenshot shows a registration form titled "Регистрация" on a dark green background. It contains three input fields: "Логин" (Login) with the text "Yarik3", "Пароль" (Password) with masked characters ".....", and "Подтверждение пароля" (Confirm password) also with masked characters ".....". Below the fields, a red error message "Пользователь уже существует" is displayed. At the bottom, there is a red button labeled "Зарегистрироваться" and a link "Уже есть аккаунт? Войти" (Already have an account? Log in).

Рисунок 5.3 – Ошибка регистрации «Пользователь уже существует»

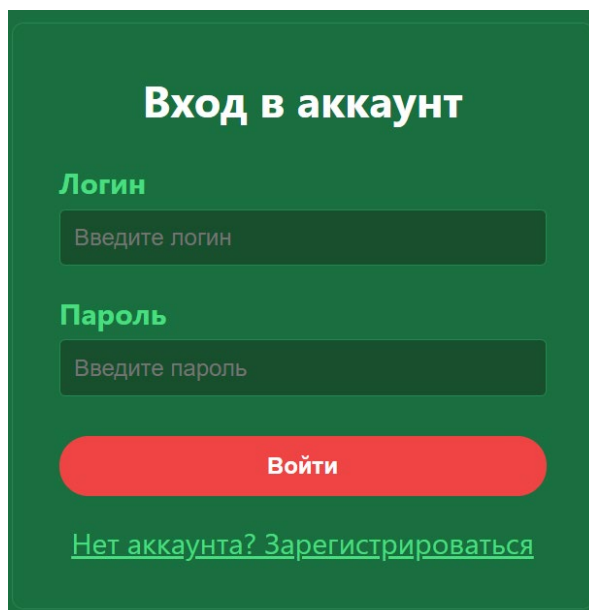
Если пользователь при регистрации ввёл неверно подтверждение пароля, то высветится ошибка «Пароли не совпадают» (рисунок 5.4)



The screenshot shows the same registration form as in Figure 5.3. The "Логин" field contains "Yarik3", the "Пароль" field contains masked characters ".....", and the "Подтверждение пароля" field contains masked characters ".....". Below the fields, a red error message "Пароли не совпадают" is displayed. At the bottom, there is a red button labeled "Зарегистрироваться" and a link "Уже есть аккаунт? Войти" (Already have an account? Log in).

Рисунок 5.4 – Ошибка регистрации

Если у пользователя уже есть аккаунт, то ему нужно нажать на кнопку «Войти» и вписать уже существующие логин и пароль от аккаунта (рисунок 5.5).



The image shows a login form with a dark green background. At the top, the title 'Вход в аккаунт' is displayed in white. Below it, the label 'Логин' is in green, followed by a dark green input field with the placeholder text 'Введите логин'. The label 'Пароль' is also in green, followed by a dark green input field with the placeholder text 'Введите пароль'. A red button with the text 'Войти' is positioned below the password field. At the bottom, there is a green link that reads 'Нет аккаунта? Зарегистрироваться'.

Рисунок 5.5 – Вход в аккаунт

После успешного входа в систему навигационная панель автоматически перестраивается. Кнопки «Войти» и «Зарегистрироваться» скрываются, и пользователю становятся доступны основная ссылка для перехода к созданию комнаты и подключению к комнате, персональный профиль, а также кнопка для выхода из учетной записи (рисунок 5.6).

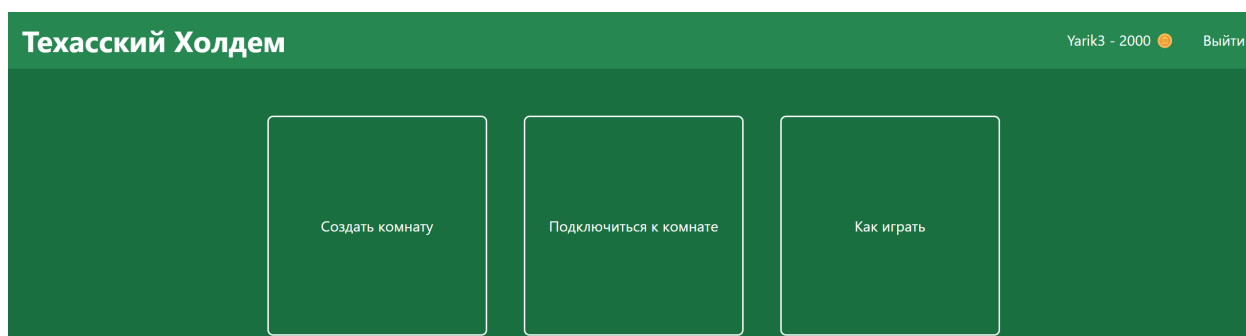


Рисунок 5.6 – Интерфейс главного меню для авторизованного пользователя

При нажатии на кнопку с пользовательским никнеймом, пользователя перебрасывают на страницу с профилем. На этой странице можно получать фишки каждый час, сменить логин, пароль и удалить аккаунт (рисунки 5.7 и 5.8).

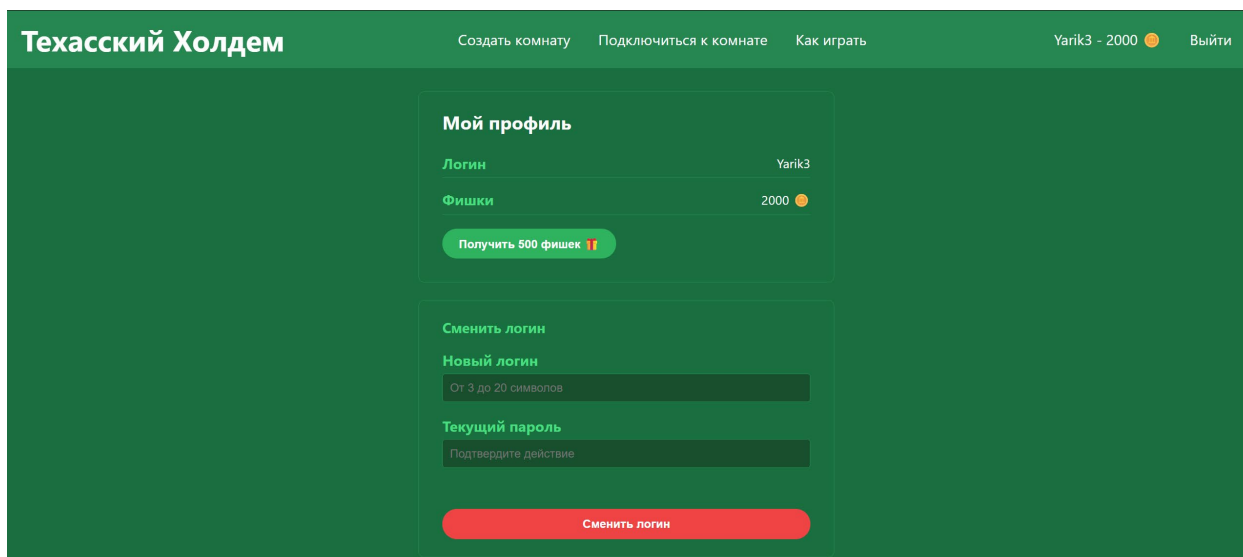


Рисунок 5.7 – Страница профиля с кнопками получения фишек и формой со сменой логина

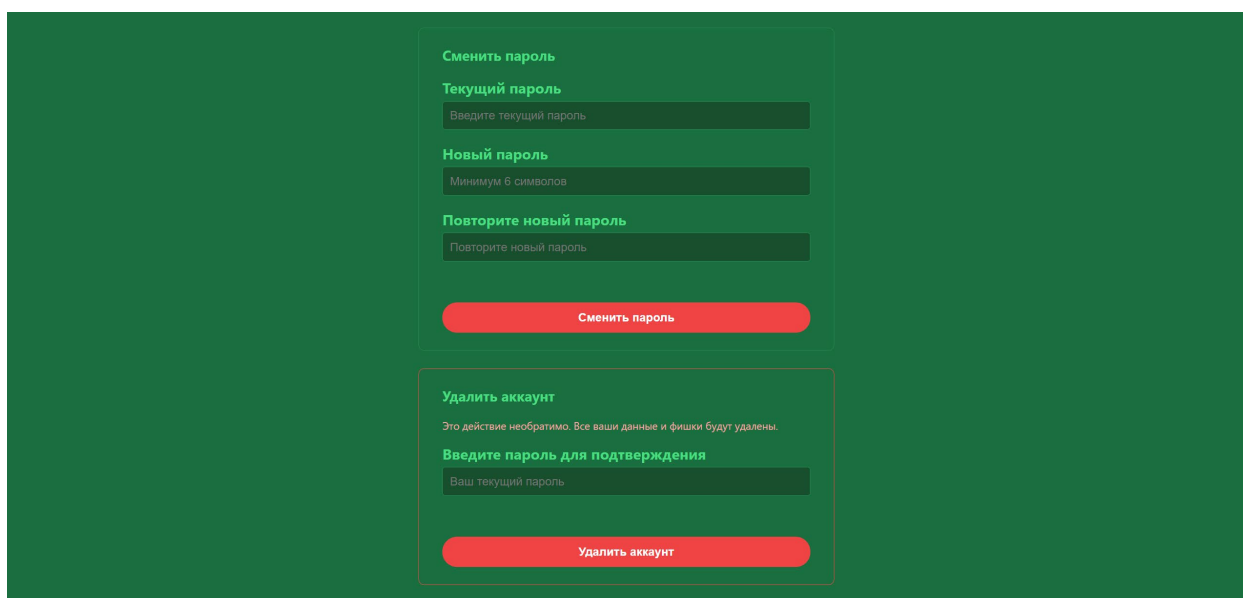


Рисунок 5.8 – Страница профиля с формами смены пароля и с удалением аккаунта

При нажатии на кнопку «Создать комнату» пользователя перебрасывают на страницу с формой по созданию комнаты. Чтобы создать комнату нужно заполнить поля формы, такие как название комнаты, максимальное количество игроков, малый и большой блайнды, минимальный бай-ин, максимальный бай и пароль комнаты (рисунок 5.9). Все поля формы кроме пароля являются обязательными.

Создание комнаты

Название комнаты
Например, 'Вечерний покер'

Максимум игроков
9

Малый блайнд
5

Большой блайнд
10

Мин. бай-ин
1000

Макс. бай-ин
3000

Пароль комнаты (необязательно)
Оставьте пустым для открытой комнаты

Создать комнату

[Подключиться к существующей комнате](#)

Рисунок 5.9 – Форма создания комнаты

Если пользователь хочет подключиться к существующей комнате, то ему нужно нажать на кнопку «Подключиться к комнате». Далее его перебросит на страницу со всеми комнатами. На этой странице пользователь может выбрать комнату, купить бай-ин в определённом промежутке и подключиться непосредственно к комнате (рисунок 5.10).

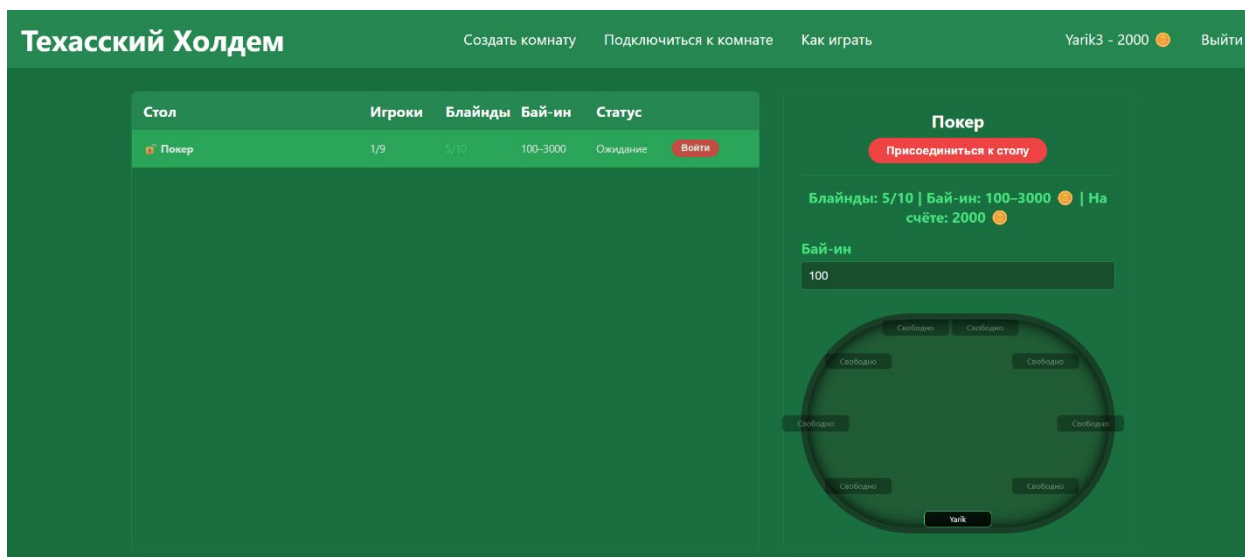


Рисунок 5.10 – Страница с подключением к комнате

При подключении к комнате появляется кнопка «Готов к раздаче», чтобы началась раздача нужно, чтобы все игроки нажали на эту кнопку (рисунок 5.11).

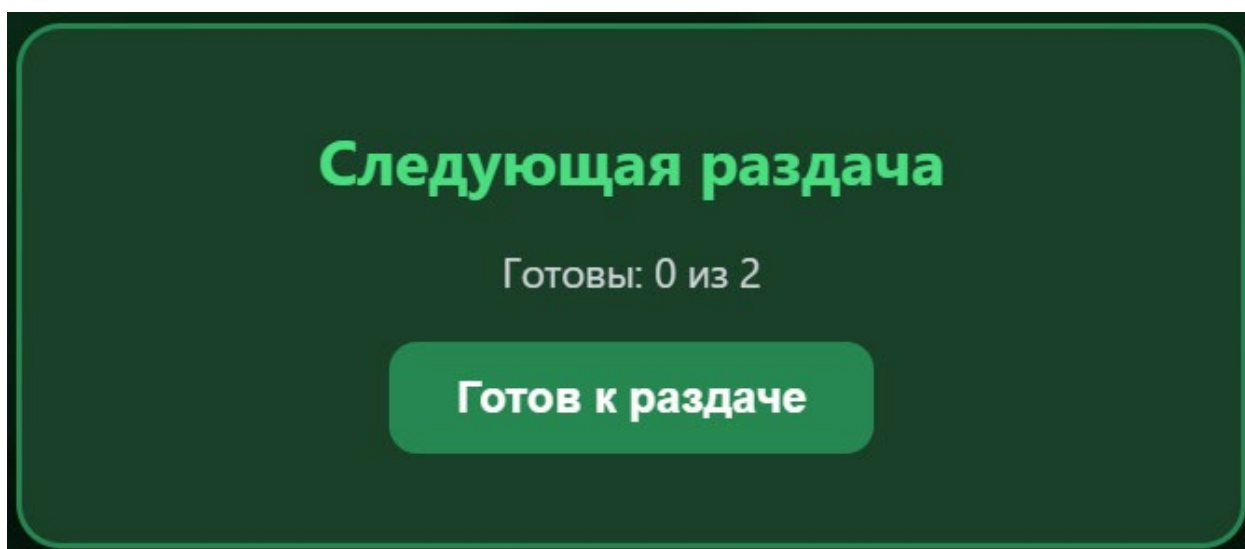


Рисунок 5.11 – Кнопка для подтверждения готовности к раздаче

После нажатия на кнопку подтверждения готовности начинается раздача. Игрокам раздаются карты, переходят блаинд-ины. На каждом этапе открываются карты и перед каждым открытием пользователь может выбрать действие: сбросить, чек, колл, ва-банк и выйти из комнаты (рисунок 5.12).

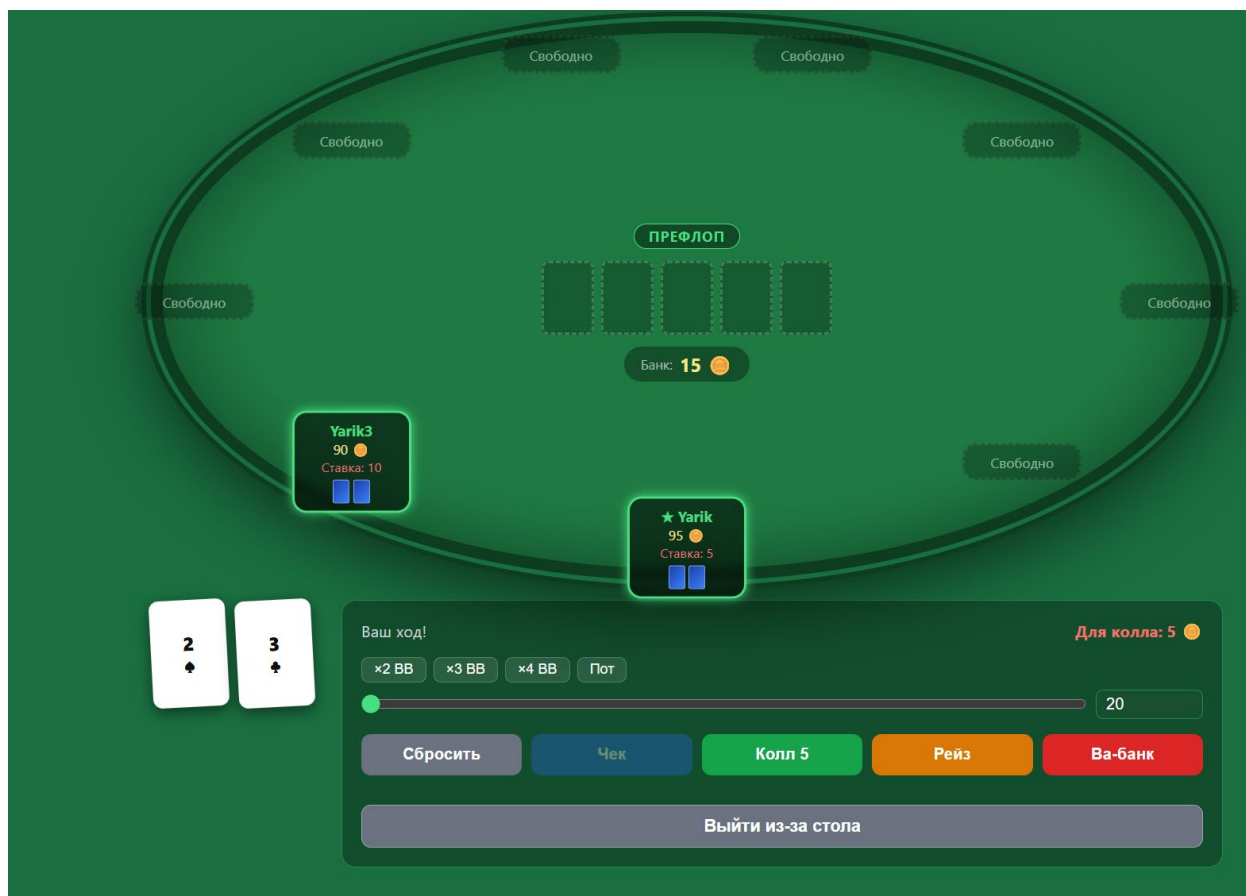


Рисунок 5.12 – Отображение стола

Победитель определяется согласно правилам покера, либо если другие игроки выходят из-за стола. В конце раздачи появляется плашка с определением победителя и его выигрышем, а также раскрываются карты всех игроков (рисунок 5.13).



Рисунок 5.13 – Конец раздачи

При выходе пользователя из-за стола, его фишки из комнаты обратно возвращаются на баланс, а самого пользователя перебрасывает в главное меню.

ЗАКЛЮЧЕНИЕ

Разработка веб-приложения для многопользовательской игры в покер «Техасский холдем» является актуальной задачей в области современных веб-технологий. В рамках данного курсового проекта было спроектировано и реализовано полнофункциональное клиент-серверное приложение, обеспечивающее проведение покерных партий в режиме реального времени через веб-браузер.

В ходе выполнения работы были решены все поставленные задачи:

- проведён анализ предметной области и выполнен сравнительный анализ существующих онлайн-платформ для игры в покер;
- спроектирована двухуровневая клиент-серверная архитектура системы;
- разработана структура приложения, включающая модули управления пользователями, игровыми комнатами, игровым процессом, сетевым взаимодействием и хранения данных;
- реализована серверная часть приложения на платформе Node.js с использованием Express, Socket.IO и PostgreSQL, обеспечивающая обработку игровых событий, управление игровыми сессиями и хранение пользовательских данных;
- разработан сетевой модуль, реализующий обмен данными между клиентом и сервером в режиме реального времени посредством Socket.IO, а также механизм обновления информации в меню с использованием технологии Server-Sent Events;
- реализована игровая логика покера «Техасский холдем», включающая раздачу карт, проведение раундов торговли, обработку игровых действий, вычисление покерных комбинаций и определение победителя раздачи;
- разработана клиентская часть приложения на основе HTML, CSS и JavaScript, обеспечивающая отображение игрового стола, управление действиями игроков и визуализацию состояния игрового процесса;
- реализована система регистрации и авторизации пользователей с использованием JWT-токенов и механизмом контроля активных пользовательских сессий;
- проведено функциональное тестирование программного средства, по результатам которого все тестовые сценарии были успешно пройдены, а выявленные в процессе разработки дефекты устранены.

Разработанное приложение полностью соответствует требованиям, сформулированным на этапе проектирования. Использование Socket.IO обеспечивает синхронизацию игрового процесса в режиме реального времени, а технология Server-Sent Events позволяет оперативно обновлять информацию в лобби. Применение PostgreSQL обеспечивает надёжное хранение пользовательских данных и игровой статистики.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- [1] Правила покера «Техасский холдем» – Poker.ru [Электронный ресурс]. – Режим доступа: <https://poker.ru/pravila-poker-tehasskij-holdem/>.
- [2] PokerStars [Электронный ресурс]. – Режим доступа: <https://www.pokerstars.com/ru/>.
- [3] 247 Free Poker [Электронный ресурс]. – Режим доступа: <https://www.247freepoker.com/>.
- [4] Replay Poker [Электронный ресурс]. – Режим доступа: <https://www.casino.org/replaypoker/ru>.
- [5] Socket.IO Documentation [Электронный ресурс]. – Режим доступа: <https://socket.io/docs/v4/>.
- [6] 18: Документация к PostgreSQL 18.4 [Электронный ресурс]. – Режим доступа: <https://postgrespro.ru/docs/postgresql/current>.
- [7] Использование отправляемых сервером событий – MDN Web Docs [Электронный ресурс]. – Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Server-sent_events/Using_server-sent_events.
- [8] Введение в REST API – RESTful веб-сервисы – Хабр [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/483202/>.
- [9] node.bcrypt.js [Электронный ресурс]. – Режим доступа: <https://www.npmjs.com/package/bcrypt>.
- [10] JWT-аутентификация в веб-приложениях – Хабр [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/842056/>.
- [11] Современный учебник JavaScript [Электронный ресурс]. – Режим доступа: <https://learn.javascript.ru/>.
- [12] CSS: Cascading Style Sheets – MDN Web Docs [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/ru/docs/Web/CSS>.
- [13] HTML: HyperText Markup Language – MDN Web Docs [Электронный ресурс]. – Режим доступа: <https://developer.mozilla.org/ru/docs/Web/HTML>.
- [14] Документация Node.js [Электронный ресурс]. – Режим доступа: <https://nodejsdev.ru/guides/webdraft/>.
- [15] Express.js Documentation [Электронный ресурс]. – Режим доступа: <https://expressjs.com/en/>.

ПРИЛОЖЕНИЕ А

Листинг кода с комментариями

Код файла server/server.js:

```
/**
 * server.js — точка входа в приложение.
 */

require('dotenv').config({ path: require('path').join(__dirname,
'.env') });

const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const jwt = require('jsonwebtoken');
const path = require('path');
const pool = require('./config/db');
const authRoutes = require('./routes/authRoutes');
const profileRoutes = require('./routes/profileRoutes');
const { socketAuth } = require('./middleware/jwtMiddleware');
const RoomManager = require('./game/roomManager');
const { router: sessionStreamRouter } = require('./routes/sessionStream');

const app = express();
const server = http.createServer(app);

// Socket.io
const io = new Server(server, {
  cors: { origin: '*', methods: ['GET', 'POST'] },
});

app.set('io', io);

// JWT-проверка для всех Socket.io соединений
io.use(socketAuth);

// Менеджер комнат — управляет игровой логикой и Socket.io
// событиями
const roomManager = new RoomManager(io);

// Делаем io доступным в контроллерах через req.app.get('io')
app.set('io', io);

// Express middleware
app.use(cors());
app.use(express.json());
app.use(cookieParser());
```

```

async function requireAuth(req, res, next) {
  const token = req.cookies.token;
  if (!token) return res.redirect('/login');
  try {
    const decoded = jwt.verify(token,
process.env.JWT_SECRET);

    // Проверяем session_token в БД
    const result = await pool.query(
      'SELECT session_token FROM users WHERE id = $1',
      [decoded.userId]
    );
    if (
      result.rows.length === 0 ||
      result.rows[0].session_token !== decoded.sessionToken
    ) {
      res.clearCookie('token');
      return res.redirect('/login');
    }

    req.user = decoded;
    next();
  } catch {
    res.clearCookie('token');
    return res.redirect('/login');
  }
}

// API маршруты
app.use('/api/auth', authRoutes);
app.use('/api/profile', profileRoutes);
app.use('/api/session', sessionStreamRouter);

// Список комнат для лобби
app.get('/api/rooms', requireAuth, (req, res) => {
  res.json(roomManager.getRoomList());
});

app.post('/api/rooms/create', requireAuth, (req, res) => {
  const result = roomManager.createRoom(req.body,
req.user.userId, req.user.username);
  if (!result.ok) return res.status(400).json(result);
  res.status(201).json(result);
});

// Защищённые HTML-маршруты — стоят ДО express.static
app.get('/create-room', requireAuth, (req, res) => {
  res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'create-room.html'));
});
app.get('/connect-to-room', requireAuth, (req, res) => {

```

```

        res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'connect-to-room.html'));
    });
    app.get('/profile', requireAuth, (req, res) => {
        res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'profile.html'));
    });
    app.get('/game', requireAuth, (req, res) => {
        res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'game.html'));
    });

    // Блокируем прямой доступ к защищённым .html файлам
    app.use('/pages', (req, res, next) => {
        if (req.path.endsWith('.html')) {
            return res.status(404).send('Страница не найдена');
        }
        next();
    });

    // Статические файлы — стоят ПОСЛЕ защищённых маршрутов
    app.use(express.static(path.join(__dirname, '..', 'client')));

    // Публичные HTML-маршруты
    app.get('/', (req, res) => {
        res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'menu.html'));
    });
    app.get('/menu', (req, res) => {
        res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'menu.html'));
    });
    app.get('/login', (req, res) => {
        res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'login.html'));
    });
    app.get('/registration', (req, res) => {
        res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'registration.html'));
    });
    app.get('/how-to-play', (req, res) => {
        res.sendFile(path.join(__dirname, '..', 'client', 'pages',
'how-to-play.html'));
    });

    // Глобальный обработчик ошибок
    // eslint-disable-next-line no-unused-vars
    app.use((err, req, res, next) => {
        console.error('Необработанная ошибка:', err);
        const statusCode = res.statusCode !== 200 ? res.statusCode :
500;

```

```

        res.status(statusCode).json({      error:      err.message      ||
'Внутренняя ошибка сервера' });
});

// Запуск
const PORT = process.env.PORT || 3000;
server.listen(PORT, () => {
    console.log(`Сервер запущен на http://localhost:${PORT}`);
});

```

Код файла server/game/RoomManager.js:

```

/**
 * RoomManager.js — менеджер покерных комнат.
 */

const GameLogic = require('./gameLogic');
const chipWallet = require('./chipWallet');
const { saveGameState, loadGameState, deleteGameState } =
require('./gameState');

const RESULT_BEFORE_READY_MS = 12000;

// Время ожидания переподключения игрока (мс).
// Если игрок не вернётся за это время — убираем со стола.
const RECONNECT_GRACE_MS = 15000;

class RoomManager {
    constructor(io) {
        this.io = io;
        this.rooms = new Map();
        this._registerSocketEvents();
        this._restoreActiveRooms();
    }

    async _restoreActiveRooms() {
        const pool = require('../config/db');
        try {
            const result = await pool.query('SELECT room_id FROM
game_states WHERE phase != $1', ['ended']);
            for (const row of result.rows) {
                await this.restoreRoom(row.room_id);
            }
            console.log(`Восстановлено      ${result.rows.length}
активных комнат`);
        } catch (err) {
            console.error('Ошибка восстановления комнат:', err);
        }
    }

    _registerSocketEvents() {
        this.io.on('connection', (socket) => {

```



```

        const playerId = socket.user.userId;
        const username = socket.user.username;

        console.log(`Socket      подключился:      ${username}
(${socket.id})`);

        socket.on('create-room', (data, callback) => {
            const result = this.createRoom(data, playerId,
username);
            callback?.(result);
        });

        socket.on('join-room', async (data, callback) => {
            const result = await this.joinRoom(socket, data,
playerId, username);
            callback?.(result);
        });

        socket.on('leave-room', async (data, callback) => {
            const result = await this.leaveRoom(socket,
data?.roomId, playerId, { force: true });
            callback?.(result);
        });

        socket.on('player-action', (data, callback) => {
            const result = this.handlePlayerAction(data,
playerId);
            callback?.(result);
        });

        socket.on('get-game-state', (data, callback) => {
            const result = this.getGameState(data?.roomId,
playerId);
            callback?.(result);
        });

        socket.on('ready', (data) => {
            this.playerReady(data?.roomId, playerId);
        });

        socket.on('disconnect', async () => {
            console.log(`Socket      отключился:      ${username}
(${socket.id})`);
            await this._handleDisconnect(socket, playerId,
username);
        });
    }

    createRoom(data, creatorId, creatorUsername) {
        const roomId =
`room_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`;
        =

```

```

const config = {
  roomName: data.roomName || 'Покерный стол',
  smallBlind: Number(data.smallBlind) || 5,
  bigBlind: Number(data.bigBlind) || 10,
  minBuyIn: Number(data.minBuyIn) || 100,
  maxBuyIn: Number(data.maxBuyIn) || 3000,
  maxPlayers: Math.min(9, Math.max(2,
Number(data.maxPlayers) || 9)),
  password: data.password || null,
};

const game = new GameLogic(config, (eventName, eventData)
=> {
  this._handleGameEvent(roomId, eventName, eventData);
});

this.rooms.set(roomId, {
  game,
  config,
  players: new Map(), // socketId → playerId
  readyPlayers: new Set(),
  autoStartTimer: null,
  disconnectTimers: new Map(), // playerId → таймер
удаления со стола
});

console.log(`Комната          создана:      ${roomId}
(${config.roomName})`);
return { ok: true, roomId, config };
}

async joinRoom(socket, data, playerId, username) {
  const { roomId, buyIn, password } = data;
  const room = this.rooms.get(roomId);

  if (!room) return { ok: false, error: 'Комната не найдена'
};
  if (room.config.password && room.config.password !==
password) {
    return { ok: false, error: 'Неверный пароль' };
  }

  // Отменяем таймер удаления — игрок переподключился
вовремя
  if (room.disconnectTimers.has(playerId)) {
    clearTimeout(room.disconnectTimers.get(playerId));
    room.disconnectTimers.delete(playerId);
    console.log(`Игрок      ${username}      переподключился
вовремя, таймер отменён`);
  }
}

```

```

// Повторное подключение — игрок уже за столом
const existing = room.game.getPlayer(playerId);
if (existing) {
    socket.join(roomId);
    room.players.set(socket.id, playerId);
    this._emitGameState(socket, room, playerId);
    const accountChips = await
chipWallet.getBalance(playerId);

    if (room.game.toAct[0]
    && String(room.game.toAct[0]) === String(playerId)
    && !['waiting', 'ended'].includes(room.game.phase)) {
        room.game._emitTurn();
    }

    // Сообщаем всем что игрок вернулся
    socket.to(roomId).emit('player-reconnected', {
playerId, username });

    return {
        ok: true,
        seatIdx: room.game.getSeatIndex(playerId),
        accountChips,
        roomInfo: this._roomInfo(room),
        reconnected: true,
    };
}

// Первый вход — списываем бай-ин
const actualBuyIn = Number(buyIn);
if (actualBuyIn < room.config.minBuyIn || actualBuyIn >
room.config.maxBuyIn) {
    return { ok: false, error: `Бай-ин должен быть от
${room.config.minBuyIn} до ${room.config.maxBuyIn}` };
}

const deduct = await chipWallet.deductChips(playerId,
actualBuyIn);
if (!deduct.ok) {
    return { ok: false, error: deduct.error, accountChips:
deduct.chips };
}

const seatIdx = room.game.sitDown(playerId, username,
actualBuyIn);
if (seatIdx === -1) {
    await chipWallet.addChips(playerId, actualBuyIn);
    const accountChips = await
chipWallet.getBalance(playerId);
    return { ok: false, error: 'Нет свободных мест',
accountChips };
}

```

```

    socket.join(roomId);
    room.players.set(socket.id, playerId);

    const inHand = !['waiting',
'ended'].includes(room.game.phase);

    this._broadcastGameState(roomId);

    socket.to(roomId).emit('player-joined-room', {
        playerId, username, seatIdx, chips: actualBuyIn,
    });

    if (inHand) {
        socket.emit('waiting-next-hand', {
            message: 'Раздача уже идёт. Вы подключитесь к
следующей после её окончания.',
        });
    } else {
        this._promptReady(roomId);
    }

    return {
        ok: true,
        seatIdx,
        accountChips: deduct.chips,
        roomInfo: this._roomInfo(room),
        handInProgress: inHand,
    };
}

async leaveRoom(socket, roomId, playerId, { force = false } =
{}) {
    const room = this.rooms.get(roomId);
    if (!room) return { ok: false, error: 'Комната не найдена'
};

    // Отменяем таймер если был
    if (room.disconnectTimers.has(playerId)) {
        clearTimeout(room.disconnectTimers.get(playerId));
        room.disconnectTimers.delete(playerId);
    }

    room.players.delete(socket.id);
    socket.leave(roomId);

    if (!force) {
        const hasOtherSocket =
[...room.players.values()].some(pid => pid === playerId);
        if (hasOtherSocket) return { ok: true };
    }
}

```

```

        let seated = room.game.getPlayer(playerId);
        if (!seated) {
            return { ok: true, accountChips: await
chipWallet.getBalance(playerId) };
        }

        const inHand = !['waiting',
'ended'].includes(room.game.phase);
        if (inHand && force) {
            room.game.forceFold(playerId);
            seated = room.game.getPlayer(playerId);
            if (!seated) {
                return { ok: true, accountChips: await
chipWallet.getBalance(playerId) };
            }
        }

        const tableChips = seated.chips;
        room.readyPlayers.delete(playerId);
        room.game.standUp(playerId);

        const cashOut = await chipWallet.addChips(playerId,
tableChips);
        socket.emit('account-chips', { chips: cashOut.chips });

        this.io.to(roomId).emit('player-left-room', { playerId
});
        this._promptReady(roomId);

        if (room.players.size === 0) {
            clearTimeout(room.autoStartTimer);
            this.rooms.delete(roomId);
            await deleteGameState(roomId);
            console.log(`Комната удалена: ${roomId}`);
        }

        return { ok: true, accountChips: cashOut.chips, cashedOut:
tableChips };
    }

    handlePlayerAction(data, playerId) {
        const { roomId, action, amount } = data;
        const room = this.rooms.get(roomId);
        if (!room) return { ok: false, error: 'Комната не найдена'
};
        return room.game.handleAction(playerId, action,
Number(amount) || 0);
    }

    getGameState(roomId, playerId) {
        const room = this.rooms.get(roomId);

```

```

        if (!room) return { ok: false, error: 'Комната не найдена'
    };
    return {
        ok: true,
        state: { ...room.game.getStateForPlayer(playerId),
roomInfo: this._roomInfo(room) },
    };
}

playerReady(roomId, playerId) {
    const room = this.rooms.get(roomId);
    if (!room) return;
    if (!['waiting', 'ended'].includes(room.game.phase))
return;

    const player = room.game.getPlayer(playerId);
    if (!player || player.chips <= 0) return;

    room.readyPlayers.add(playerId);

    const eligible = room.game.getOccupiedSeats().filter(p =>
p.chips > 0);
    const allCount = eligible.length;
    const readyCount = [...room.readyPlayers].filter(id =>
        eligible.some(p => p.id === id)
    ).length;

    this.io.to(roomId).emit('player-ready', { playerId,
readyCount, allCount });

    if (readyCount >= allCount && allCount >= 2) {
        this._startHand(roomId);
    }
}

_handleGameEvent(roomId, eventName, eventData) {
    if (eventName === 'deal-hole-cards') {
        const { targetPlayerId, holeCards } = eventData;
        const socketId = this._getSocketId(roomId,
targetPlayerId);
        if (socketId) {
            this.io.to(socketId).emit('hole-cards', {
holeCards });
        }
        return;
    }

    this.io.to(roomId).emit(eventName, eventData);

    if (eventName === 'player-joined' || eventName ===
'player-left') {
        this._broadcastGameState(roomId);
    }
}

```

```

    }

    if (eventName === 'showdown' || eventName === 'hand-ended-
no-showdown') {
        const room = this.rooms.get(roomId);
        if (!room) return;
        clearTimeout(room.autoStartTimer);
        room.autoStartTimer = setTimeout(() => {
            this._promptReady(roomId);
        }, RESULT_BEFORE_READY_MS);
    }

    // Автосохранение после каждого события
    this._saveRoomState(roomId);
}

_promptReady(roomId) {
    const room = this.rooms.get(roomId);
    if (!room) return;

    clearTimeout(room.autoStartTimer);
    room.readyPlayers.clear();

    const eligible = room.game.getOccupiedSeats().filter(p =>
p.chips > 0);

    if (eligible.length < 2) {
        this.io.to(roomId).emit('waiting-for-players', {
            message: 'Ждём минимум 2 игроков с фишками...',
        });
        return;
    }

    if (!['waiting', 'ended'].includes(room.game.phase))
return;

    this.io.to(roomId).emit('awaiting-ready', {
        readyCount: 0,
        allCount: eligible.length,
    });
}

_startHand(roomId) {
    const room = this.rooms.get(roomId);
    if (!room) return;
    const readyIds = [...room.readyPlayers];
    room.readyPlayers.clear();
    const started = room.game.startHand(readyIds);
    if (!started) {
        this.io.to(roomId).emit('waiting-for-players', {
            message: 'Ждём минимум 2 игроков с фишками...',
        });
    }
}

```

```

    }
  }

  /**
   * При дисконнекте даём игроку RECONNECT_GRACE_MS миллисекунд
   * на переподключение. Если не вернулся — убираем со стола.
   */
  async _handleDisconnect(socket, playerId, username) {
    for (const [roomId, room] of this.rooms.entries()) {
      if (!room.players.has(socket.id)) continue;

      room.players.delete(socket.id);

      if (socket.sessionKicked) {
        console.log(`[Session Kick] ${username} убирается
из комнаты ${roomId} немедленно`);
        await this.leaveRoom(socket, roomId, playerId, {
force: true });
        continue;
      }

      // Обычный дисконнект (обрыв связи) — даём 15 секунд
на переподключение
      this.io.to(roomId).emit('player-temporarily-
disconnected', {
        playerId,
        username,
        reconnectMs: RECONNECT_GRACE_MS,
      });
      console.log(`${username} отключился, ждём
${RECONNECT_GRACE_MS / 1000}с...`);

      const timer = setTimeout(async () => {
        room.disconnectTimers.delete(playerId);
        console.log(`${username} не переподключился, убираем
со стола`);
        const ghostSocket = {
          id: `ghost_${playerId}`,
          emit: () => {},
          leave: () => {},
        };
        await this.leaveRoom(ghostSocket, roomId, playerId, {
force: true });
        }, RECONNECT_GRACE_MS);

      room.disconnectTimers.set(playerId, timer);
    }
  }

  _roomInfo(room) {
    return {
      roomName: room.config.roomName,

```



```

        smallBlind: room.config.smallBlind,
        bigBlind: room.config.bigBlind,
    };
}

_emitGameState(socket, room, playerId) {
    const state = room.game.getStateForPlayer(playerId);
    socket.emit('game-state', { ...state, roomInfo:
this._roomInfo(room) });
}

_broadcastGameState(roomId) {
    const room = this.rooms.get(roomId);
    if (!room) return;
    for (const [socketId, pid] of room.players.entries()) {
        const socket = this.io.sockets.sockets.get(socketId);
        if (socket) this._emitGameState(socket, room, pid);
    }
}

_getSocketId(roomId, playerId) {
    const room = this.rooms.get(roomId);
    if (!room) return null;
    for (const [socketId, pid] of room.players.entries()) {
        if (pid === playerId) return socketId;
    }
    return null;
}

getRoomList() {
    const list = [];
    for (const [roomId, room] of this.rooms.entries()) {
        const players = room.game.getOccupiedSeats();
        list.push({
            roomId,
            roomName: room.config.roomName,
            playerCount: players.length,
            seats: room.game.seats.map(s => (s ? { username:
s.username } : null)),
            maxPlayers: room.config.maxPlayers,
            smallBlind: room.config.smallBlind,
            bigBlind: room.config.bigBlind,
            minBuyIn: room.config.minBuyIn,
            maxBuyIn: room.config.maxBuyIn,
            hasPassword: !!room.config.password,
            phase: room.game.phase,
        });
    }
    return list;
}

async _saveRoomState(roomId) {

```

```

const room = this.rooms.get(roomId);
if (!room) return;

try {
  await saveGameState(roomId, {
    config: room.config,
    seats: room.game.seats,
    phase: room.game.phase,
    communityCards: room.game.communityCards,
    dealerSeat: room.game.dealerSeat,
    currentSeat: room.game.currentSeat,
    currentBet: room.game.currentBet,
    streetBets: room.game.streetBets,
    totalContributed: room.game.totalContributed,
    deck: room.game.deck,
    toAct: room.game.toAct,
    lastRaiseAmount: room.game.lastRaiseAmount,
    actedThisStreet: [...room.game.actedThisStreet]
  });
} catch (err) {
  console.error(`Ошибка сохранения состояния комнаты
${roomId}:`, err);
}

async restoreRoom(roomId) {
  const state = await loadGameState(roomId);
  if (!state) return null;

  const game = new GameLogic(state.config, (eventName,
eventData) => {
    this._handleGameEvent(roomId, eventName, eventData);
  });

  game.seats = state.seats;
  game.phase = state.phase;
  game.communityCards = state.communityCards;
  game.dealerSeat = state.dealerSeat;
  game.currentSeat = state.currentSeat;
  game.currentBet = state.currentBet;
  game.streetBets = state.streetBets;
  game.totalContributed = state.totalContributed;
  game.deck = state.deck;
  game.toAct = state.toAct;
  game.lastRaiseAmount = state.lastRaiseAmount ||
state.config.bigBlind;
  game.actedThisStreet = new Set(state.actedThisStreet ||
[]);

  this.rooms.set(roomId, {
    game,
    config: state.config,

```

```

        players: new Map(),
        readyPlayers: new Set(),
        autoStartTimer: null,
        disconnectTimers: new Map(),
    });

    // Если игра в активной фазе и есть очередь ходов —
    отправляем событие о текущем ходе
    if (game.toAct.length > 0 && ![ 'waiting',
    'ended' ].includes(game.phase)) {
        game._emitTurn();
    }

    console.log(`Комната восстановлена: ${roomId}`);
    return { ok: true, roomId };
}

module.exports = RoomManager;

```

Код файла server/middleware/jwtMiddleware.js:

```

/**
jwtMiddleware.js — middleware для проверки JWT-токена.
Содержит две версии:
expressAuth — для защиты HTTP-маршрутов Express
socketAuth — для защиты Socket.io соединений
После успешной проверки данные пользователя доступны как:
req.user      (в Express-маршрутах)
socket.user    (в Socket.io обработчиках)
*/
const jwt = require('jsonwebtoken');
const pool = require('../config/db');

/**
Middleware для Express.
Ожидает заголовок: Authorization: Bearer <token>
Пример защищённого маршрута:
app.get('/api/profile', expressAuth, (req, res) => {
    res.json(req.user); // { userId, username }
});
*/
async function expressAuth(req, res, next) {
    try {
        let token;
        const authHeader = req.headers['authorization'];
        if (authHeader && authHeader.startsWith('Bearer ')) {
            token = authHeader.slice(7);
        } else if (req.cookies?.token) {
            // Запасной вариант — читаем из httpOnly куки (для
            браузерных запросов)
            token = req.cookies.token;

```

```

    }

    if (!token) {
        return res.status(401).json({ error: 'Токен
отсутствует или имеет неверный формат' });
    }

    const decoded = jwt.verify(token,
process.env.JWT_SECRET);

    // Проверка валидности сессии в БД (защита от
одновременных входов)
    const result = await pool.query('SELECT session_token FROM
users WHERE id = $1', [decoded.userId]);
    if (result.rows.length === 0 ||
result.rows[0].session_token !== decoded.sessionToken) {
        res.clearCookie('token');
        return res.status(401).json({ error: 'Сессия истекла:
вход выполнен с другого устройства' });
    }

    req.user = {
        userId: decoded.userId,
        username: decoded.username,
    };
    next();
} catch (err) {
    res.clearCookie('token');
    return res.status(401).json({ error: 'Токен
недействителен или истёк' });
}
}

/**
Middleware для Socket.io.
Подключается через: io.use(socketAuth)
Ожидает токен в: socket.handshake.auth.token
Пример на клиенте:
const socket = io({ auth: { token: localStorage.getItem('token')
} });
После успешной проверки в обработчиках доступно socket.user.
*/
async function socketAuth(socket, next) {
    try {
        const token = socket.handshake.auth?.token;
        if (!token) {
            // next(new Error(...)) – стандартный способ отклонить
соединение в Socket.io
            return next(new Error('Authentication error: токен не
передан'));
        }
    }

```

```

        const decoded = jwt.verify(token,
process.env.JWT_SECRET);

        // Проверка валидности сессии в БД для защиты от
одновременных сессий
        const result = await pool.query('SELECT session_token FROM
users WHERE id = $1', [decoded.userId]);
        if (result.rows.length === 0 ||
result.rows[0].session_token !== decoded.sessionToken) {
            return next(new Error('Session expired: вход выполнен
с другого устройства'));
        }

        // Сохраняем данные пользователя в объект сокета
        socket.user = {
            userId: decoded.userId,
            username: decoded.username,
        };

        next(); // Разрешаем соединение
    } catch (err) {
        return next(new Error('Authentication error: токен
недействителен или истёк'));
    }
}

module.exports = { expressAuth, socketAuth };

```

Код файла client/js/mainGame.js:

```

/**
 * Главный файл игры — подключение к серверу и обработка событий
 */

import { state, PHASE_NAMES, HAND_PHASES } from './game/state.js';
import { renderPlayers, renderEmptySeat, findSeatByPlayerId,
renderSeatAtShowdown, renderSeatShowdownCards,
syncMyTableChipsFromPlayers } from './game/players.js';
import { renderHoleCards, renderCommunityCards, clearHandCards,
clearHoleCards } from './game/cards.js';
import { renderTurnUI, disableAllActions, setupActionButton } from
'./game/actions.js';
import { showResult, dismissResultOverlay, updateReadyUI,
resetReadyButton, showOverlay, hideOverlay, setActionPrompt,
setWaitingForPlayersStatus, updateIdlePrompt } from
'./game/ui.js';
import { updatePot, updatePhase, clearSidePots, updateRoomHeader
} from './game/pot.js';

const urlParams = new URLSearchParams(window.location.search);
state.roomId = urlParams.get('roomId');

```

```

if (!state.roomId) {
  alert('Комната не указана');
  window.location.href = '/connect-to-room';
}

const token = localStorage.getItem('token');
const currentUser = JSON.parse(localStorage.getItem('currentUser') || '{}');
state.myPlayerId = currentUser.id;
state.myUsername = currentUser.username;

if (!token) {
  window.location.href = '/login';
}

const socket = io({ auth: { token } });

socket.on('connect', () => {
  const joiningData = JSON.parse(localStorage.getItem('joiningRoom') || '{}');

  socket.emit('join-room', {
    roomId: state.roomId,
    buyIn: joiningData.buyIn || 1000,
    password: joiningData.password || null,
  }, (res) => {
    if (!res?.ok) {
      if (res?.accountChips !== null)
        syncAccountChips(res.accountChips);
      return;
    }

    if (res.accountChips !== null)
      syncAccountChips(res.accountChips);
    if (res.roomInfo) updateRoomHeader(res.roomInfo);
    if (!res.reconnected && joiningData.buyIn) {
      state.myTableChips = joiningData.buyIn;
    }

    localStorage.removeItem('joiningRoom');
    if (res.handInProgress) {
    }

    socket.emit('get-game-state', { roomId: state.roomId },
    (stateRes) => {
      if (stateRes?.ok) renderGameState(stateRes.state);
    });
  });
});

socket.on('connect_error', (err) => {
});

```

```

socket.on('disconnect', () => {
    disableAllActions();
});

socket.on('session-kicked', (data) => {
    socket.disconnect();
    localStorage.removeItem('token');
    localStorage.removeItem('currentUser');
    // Для игры лучше сразу редиректить без alert, чтобы не ломать
    UI
    window.location.href = '/login';
});

socket.on('account-chips', ({ chips }) => {
    syncAccountChips(chips);
});

socket.on('game-state', (gameState) => {
    renderGameState(gameState);
});

socket.on('hole-cards', ({ holeCards }) => {
    renderHoleCards(holeCards);
});

socket.on('hand-started', ({ dealerSeat, phase, players }) => {
    state.isSpectating = false;
    syncMyTableChipsFromPlayers(players);
    clearHandCards();
    clearSidePots();
    updatePhase(phase);
    renderPlayers(players, phase);
    setActionPrompt('Ожидайте своего хода...');
    dismissResultOverlay();
    hideOverlay('overlay-ready');
    resetReadyButton();
});

socket.on('blinds-posted', ({ sbPlayerId, bbPlayerId, smallBlind,
bigBlind }) => {
    state.bigBlind = bigBlind;
});

socket.on('street-changed', ({ phase, communityCards }) => {
    updatePhase(phase);
    renderCommunityCards(communityCards);
    disableAllActions();
    document.getElementById('action-prompt').textContent =
'Oжидайте своего хода...';
    document.getElementById('to-call-display').textContent = '';
});

```

```

socket.on('player-turn', (data) => {
  updatePot(data.pot);
  if (data.players) {
    renderPlayers(data.players);
    syncMyTableChipsFromPlayers(data.players);
  }
  renderTurnUI(data);
});

socket.on('player-action', ({ playerId, username, action, amount,
chips, pot, players }) => {
  updatePot(pot);
  if (players) {
    renderPlayers(players);
    syncMyTableChipsFromPlayers(players);
  } else if (String(playerId) === String(state.myPlayerId) &&
chips != null) {
    state.myTableChips = chips;
  }
  const actionLabels = {
    fold: 'сбрасывает',
    check: 'чекает',
    call: `коллирует ${amount} □`,
    raise: `рейзит до ${amount} □`,
    allIn: `идёт ва-банк (${amount} □)`,
  };
});

socket.on('player-joined', ({ seatIdx, username, chips }) => {
});

socket.on('player-left', ({ seatIdx, playerId }) => {
  renderEmptySeat(seatIdx);
  updateIdlePrompt();
});

socket.on('player-temporarily-disconnected', ({ username }) => {
});

socket.on('player-reconnected', ({ username }) => {
});

socket.on('waiting-for-players', ({ message }) => {
  dismissResultOverlay();
  hideOverlay('overlay-ready');
  setWaitingForPlayersStatus();
});

socket.on('waiting-next-hand', ({ message }) => {
  state.isSpectating = true;
  disableAllActions();
});

```



```

        hideOverlay('overlay-ready');
        const prompt = document.getElementById('action-prompt');
        if (prompt) prompt.textContent = 'Ожидаете следующую
раздачу...';
    });

socket.on('awaiting-ready', ({ readyCount, allCount }) => {
    dismissResultOverlay();
    resetReadyButton();
    updateReadyUI(readyCount, allCount);
    setActionPrompt('Подтвердите готовность к раздаче');
    showOverlay('overlay-ready');
});

socket.on('player-ready', ({ playerId, readyCount, allCount }) => {
    {
        updateReadyUI(readyCount, allCount);
        if (String(playerId) === String(state.myPlayerId)) {
            state.isReady = true;
            const btn = document.getElementById('btn-ready');
            if (btn) {
                btn.disabled = true;
                btn.textContent = 'ВЫ ГОТОВЫ';
            }
        }
        if (readyCount >= allCount && allCount >= 2) {
            hideOverlay('overlay-ready');
        }
    }
});

socket.on('hand-ended-no-showdown', ({ winnerId, winnerUsername,
amount, players }) => {
    updatePhase('ended');
    clearHandCards();
    renderPlayers(players, 'ended');
    syncMyTableChipsFromPlayers(players);
    updatePot(0);
    clearSidePots();
    showResult([
        {
            username: winnerUsername,
            amount,
            handName: null,
            isWinner: true,
        }, false);
    disableAllActions();
    updateIdlePrompt();
});

socket.on('showdown', ({ evaluations, pots, results,
communityCards, players }) => {
    updatePhase('showdown');
    renderCommunityCards(communityCards);

```

```

    updatePot(0);
    clearSidePots();
    if (players) {
        syncMyTableChipsFromPlayers(players);
        state.tablePlayers = players;
        renderPlayers(players, 'showdown');
    }

    revealAllPlayersCards(evaluations);

    const resultItems = results.map(r => ({
        username: r.username,
        amount: r.amount,
        handName: r.hand?.name,
        isWinner: r.amount > 0,
    }));
    showResult(resultItems, true);

    disableAllActions();
    state.phase = 'ended';
    updatePhase('ended');
});

function renderGameState(gs) {
    if (!gs) return;

    if (gs.roomInfo) updateRoomHeader(gs.roomInfo);
    state.phase = gs.phase || 'waiting';
    updatePhase(gs.phase);
    updatePot(gs.pot);

    const activePhases = ['preflop', 'flop', 'turn', 'river',
'showdown'];
    if (activePhases.includes(gs.phase)) {
        hideOverlay('overlay-ready');
    }

    if (gs.communityCards?.length) {
        renderCommunityCards(gs.communityCards);
    }
    if (gs.players) {
        renderPlayers(gs.players, gs.phase);
        syncMyTableChipsFromPlayers(gs.players);
    }

    applySpectatorMode(gs);
    updateIdlePrompt(gs.players);

    const myPlayerData = gs.players?.find(
        p => p?.id !== null && String(p.id) ===
String(state.myPlayerId)
    );

```

```

    if (HAND_PHASES.includes(gs.phase)
        && myPlayerData?.holeCards?.length
        && !myPlayerData.holeCards[0].hidden) {
        renderHoleCards(myPlayerData.holeCards);
    } else {
        clearHoleCards();
    }
}

function applySpectatorMode(gs) {
    const me = gs.players?.find(
        p => p?.id !== null && String(p.id) ===
String(state.myPlayerId)
    );
    const spectating = HAND_PHASES.includes(gs.phase) &&
me?.status === 'out';
    state.isSpectating = spectating;

    if (spectating) {
        disableAllActions();
        hideOverlay('overlay-ready');
        const prompt = document.getElementById('action-prompt');
        if (prompt) prompt.textContent = 'Ожидаете следующую
раздачу...';
    } else if (HAND_PHASES.includes(gs.phase)) {
        state.isSpectating = false;
    }
}

function revealAllPlayersCards(evaluations) {
    if (!evaluations?.length) return;

    for (const ev of evaluations) {
        const seatIdx = resolveSeatIdx(ev);
        if (seatIdx === null || seatIdx < 0) continue;

        const player = state.tablePlayers?.[seatIdx];
        if (player) {
            renderSeatAtShowdown(seatIdx, player, ev);
        } else {
            renderSeatShowdownCards(seatIdx, ev.holeCards,
ev.handName);
        }
    }

    const myEv = evaluations.find(e => String(e.playerId) ===
String(state.myPlayerId));
    if (myEv?.holeCards?.length) {
        renderHoleCards(myEv.holeCards);
        const handNameEl = document.getElementById('hand-name');
        if (handNameEl && myEv.handName) handNameEl.textContent =
myEv.handName;
    }
}

```

```

    }
}

function resolveSeatIdx(ev) {
    if (ev.seatIdx !== null && ev.seatIdx >= 0) return ev.seatIdx;
    return findSeatByPlayerId(ev.playerId);
}

function syncAccountChips(chips) {
    const stored = JSON.parse(localStorage.getItem('currentUser')
|| '{}');
    stored.chips = chips;
    localStorage.setItem('currentUser', JSON.stringify(stored));
    renderProfileNav();
}

function renderProfileNav() {
    const profileNav = document.querySelector('.profile-nav ul');
    if (!profileNav) return;
    const user = JSON.parse(localStorage.getItem('currentUser')
|| '{}');
    if (!user?.username) return;

    const account = user.chips ?? 0;

    profileNav.innerHTML = `
        <li><a href="/profile" id="profile-menu-link"
title="Баланс на счёте">${user.username} - ${account} ☐

```

```

        const res = await fetch('/api/profile/me', { credentials:
'same-origin' });
        if (!res.ok) return;
        const user = await res.json();
        syncAccountChips(user.chips);
        const stored =
JSON.parse(localStorage.getItem('currentUser') || '{}');
        stored.id = user.id;
        stored.username = user.username;
        localStorage.setItem('currentUser',
JSON.stringify(stored));
        renderProfileNav();
    } catch {
        renderProfileNav();
    }
}

const LEAVE_TABLE_CONFIRM = 'Выйти из-за стола? Все фишки со стола
вернутся на ваш счёт.';

function confirmLeaveTable() {
    return confirm(LEAVE_TABLE_CONFIRM);
}

function leaveRoom(onSuccess) {
    socket.emit('leave-room', { roomId: state.roomId }, (res) =>
{
        if (res?.ok === false) {
            alert(res.error || 'Не удалось выйти из-за стола');
            return;
        }
        if (res?.accountChips !== null)
syncAccountChips(res.accountChips);
        state.myTableChips = 0;
        socket.disconnect();
        onSuccess?.();
    });
}

function leaveTableAndGo(targetUrl = '/menu') {
    leaveRoom(() => { window.location.href = targetUrl; });
}

function leaveTableWithConfirm(targetUrl = '/menu') {
    if (!confirmLeaveTable()) return;
    leaveTableAndGo(targetUrl);
}

document.getElementById('btn-ready')?.addEventListener('click',
() => {
    if (state.isReady || state.isSpectating) return;
    socket.emit('ready', { roomId: state.roomId });

```

```

    state.isReady = true;
    const btn = document.getElementById('btn-ready');
    if (btn) {
        btn.disabled = true;
        btn.textContent = 'ВЫ ГОТОВЫ';
    }
});

document.getElementById('btn-leave-
table')?.addEventListener('click', () => {
    leaveTableWithConfirm('/menu');
});

document.querySelector('header h1 a')?.addEventListener('click',
(e) => {
    e.preventDefault();
    leaveTableWithConfirm('/menu');
});

setupActionButtons(socket);
refreshProfileNav();

```

Код файла server/routes/sessionStream.js:

```

const express = require('express');
const jwt = require('jsonwebtoken');
const router = express.Router();

// Хранилище активных SSE-соединений: Map<userId(string),
Set<res>>
const sseClients = new Map();

/**
 * Отправить событие конкретному пользователю по всем его активным
SSE-соединениям.
 */
function notifyUser(userId, event, data) {
    const clients = sseClients.get(String(userId));
    if (!clients || clients.size === 0) return;
    const payload = `event: ${event}\ndata:
${JSON.stringify(data)}\n\n`;
    for (const res of clients) {
        try { res.write(payload); } catch {}
    }
}

// GET /api/session/stream
router.get('/stream', (req, res) => {
    const token = req.cookies?.token;
    if (!token) return res.sendStatus(401);

    let decoded;

```

```

try {
  decoded = jwt.verify(token, process.env.JWT_SECRET);
} catch {
  return res.sendStatus(401);
}

const userId = String(decoded.userId);

// Заголовки SSE
res.setHeader('Content-Type', 'text/event-stream');
res.setHeader('Cache-Control', 'no-cache');
res.setHeader('Connection', 'keep-alive');
res.setHeader('X-Accel-Buffering', 'no');
res.flushHeaders();

// Подтверждение, что поток открыт
res.write('event: connected\n');

if (!sseClients.has(userId)) sseClients.set(userId, new Set());
sseClients.get(userId).add(res);

req.on('close', () => {
  const set = sseClients.get(userId);
  if (!set) return;
  set.delete(res);
  if (set.size === 0) sseClients.delete(userId);
});
});

// Правильный экспорт для CommonJS
module.exports = { router, notifyUser };

```

Код файла server/config/db.js:

```

/**
 * db.js — модуль подключения к PostgreSQL.
 * Использует пул клиентов (pg.Pool) для эффективного
 * переиспользования соединений в многопользовательской среде.
 * Параметры подключения берутся из переменных окружения (.env).
 */

const { Pool } = require('pg');

// Создаём пул соединений. pg.Pool автоматически управляет
// жизненным циклом соединений: открывает новые при нагрузке,
// закрывает простаивающие. Для учебного проекта хватит 10
// соединений.
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  // Опционально: максимальное количество соединений в пуле
  max: 10,
  // Время ожидания соединения из пула (мс)

```

```

        idleTimeoutMillis: 30000,
        // Время ожидания установки нового соединения (мс)
        connectionTimeoutMillis: 2000,
    });

    // Проверяем подключение при старте — если БД недоступна,
    // сразу получим ошибку вместо первого запроса пользователя
    pool.connect((err, client, release) => {
        if (err) {
            console.error('Ошибка подключения к PostgreSQL:',
err.message);
        } else {
            console.log('Подключение к PostgreSQL установлено');
            release(); // Возвращаем клиента обратно в пул
        }
    });

module.exports = pool;

```

Код файла server/routes/authRoutes.js:

```

/**
 * authRoutes.js — маршруты аутентификации.
 */

const express = require('express');
const router = express.Router();
const { register, login, logout } = require('../controllers/authController');

// POST /api/auth/register — регистрация
router.post('/register', register);

// POST /api/auth/login — вход, получение токена и установка куки
router.post('/login', login);

// POST /api/auth/logout — выход, очистка куки
router.post('/logout', logout);

module.exports = router;

```


ПРИЛОЖЕНИЕ Б
Схема алгоритма работы системы